

第8章: 編集・削除・詳細表示機能の実装

前章で作った「一覧表示（Read）」と「新規登録（Create）」に続いて、この章では残りの機能を全て実装します。この章を終えると、**CRUDの全機能が揃った完動するアプリ**が完成します！

この章で実装する機能

前章までで、書籍の一覧表示と新規登録機能を実装しました。この章では、CRUD（Create=作成, Read=読み取り, Update=更新, Delete=削除）の残りの機能を実装します。

機能	CRUDのどれ？	ユーザーの操作	技術的に行うこと
詳細表示	R (Read)	書籍カードをクリック → 全情報を表示	動的ルーティング（ <code>/books/[id]</code> ）で Supabase から1件取得
編集	U (Update)	詳細画面の「編集」ボタン → フォームで修正 → 保存	既存データをフォームに表示し、Supabase の UPDATE を実行
削除	D (Delete)	「削除」ボタン → 確認ダイアログ → 削除	Supabase の DELETE を実行し、一覧に戻る
検索・フィルタ	(発展)	キーワード入力やステータスで絞り込み	Supabase の <code>ilike</code> （部分一致検索）クエリを使用
ソート	(発展)	「新しい順」「評価順」などで並べ替え	Supabase の <code>order</code> クエリを使用

動的ルーティングとは？ URLの一部を変数として扱う仕組みです。例えば `/books/abc123` と

`/books/xyz789` は、同じページ定義（`/books/[id]/page.tsx`）で処理されます。`[id]` の部分がそれぞれ `abc123` や `xyz789` に置き換わります。「URLの中の変わる部分」を **動的セグメント（dynamic segment）** と呼びます。セグメントとは URL を `/` で区切った 1 区画のことで、`/books/abc123` なら `books` と `abc123` がそれぞれ 1 セグメントになります。

HTTP メソッドのざっくり整理: ブラウザがサーバーに対して行う操作には種類があり、これを HTTP メソッドと呼びます。CRUD と対応付けると次のようになります。 - **GET:** ページやデータを「読む」（Read）。一覧表示や詳細表示で使う。何度実行しても結果が変わらない。 - **POST:** 新しいデータを「作る」（Create）。フォーム送信などで使う。実行のたびに新しいレコードが増える。 - **PUT / PATCH:** 既存データを「書き換える」（Update）。同じリクエストを 2 回送っても結果が同じ（これを **冪等性（べきとうせい）** と呼びます）。 - **DELETE:** データを「消す」（Delete）。一度消したものをもう一度消そうとしても、すでに無いので結果は同じ（これも冪等）。

Next.js では、これらを意識せずに JavaScript の関数として呼び出せる場面が多いですが、裏側ではこの 4 つのいずれかが走っています。

目次

0. 前提知識: 動的ルーティング・useState・useRouter

1. 詳細表示機能
 2. 編集機能
 3. 削除機能
 4. 検索・フィルタ機能（発展）
 5. ソート機能
 6. 全機能の動作確認
 7. トラブルシューティング
 8. 全体のページ遷移図
-

0. 前提知識: 動的ルーティング・useState・useRouter

この章では、特定の本を「ID指定で開く・編集する・削除する」機能を作ります。コードに入る前に、3つの仕組みだけ押さえます。

0.1 動的ルーティング（Dynamic Routes）

「URLの中に変数を含めたい」場合に使う Next.js の機能です。

```
app/books/[id]/page.tsx ← フォルダ名を【角括弧】で囲む
```

このファイルがあると、

アクセスされたURL	内部で <code>params.id</code> の値
<code>/books/abc123</code>	<code>"abc123"</code>
<code>/books/xyz789</code>	<code>"xyz789"</code>
<code>/books/42</code>	<code>"42"</code>

のように、`[id]` の部分が変数として取り出せます。Next.js 15 以降では `params` は `Promise` で渡されるため、`await` で取り出す必要があります（後ほどコード内で出てきます）。

なぜ Promise なの？ Next.js 15 から、ページコンポーネントの `params` や `searchParams` は「すぐには値が確定していないかもしれない非同期データ」として扱われるようになりました。Promise とは「あとで値が決まる箱」のようなもので、`await` を付けることで「箱から値が出てくるのを待つ」ことができます。Next.js 14 以前は普通のオブジェクトだったため、移行時にこの違いでつまづきやすいので注意してください。

0.2 `useState` の超復習

第3章で出てきた React Hook。「コンポーネントが覚えておきたい状態（変化する値）」を持つために使います。

▼ このコードがやること（先に日本語で）：ボタンを押すたびに数字が増える、シンプルなカウンターを作ります。カギになるのは `useState` で、「画面に覚えさせておきたい値（ここではカウント数）」と「その値を書き換える専用の関数」をセットで受け取ります。初心者はまず「値を直接書き換えるのではなく、更新用の関数を呼ぶと画面が描き直される」という流れだけ押さえてください。各行の詳しい意味はコード内のコメントにあります。

```
// このファイルはクライアント側（ブラウザ）で動かす宣言。これがないと useState などのフックは使えない
"use client";
// React から useState フックを取り込む
import { useState } from "react";

// Counter という関数コンポーネントを定義し、デフォルトエクスポートする
export default function Counter() {
  // [現在の値, 値を更新する関数] = useState(初期値)
  // 配列分割代入で 2 つの値を一度に受け取る書き方
  // count の初期値は 0、setCount で値を更新する
  const [count, setCount] = useState(0);

  return (
    <div>
      {/* 波括弧 { } の中に JavaScript の値を書くと、その値が描画される */}
      <p>カウント: {count}</p>
      {/* クリック時に count を +1 する関数を渡す */}
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

▼ 動作: - 最初に画面に「カウント: 0」と表示される - ボタンを押すたびに count が +1 され、画面が再描画される

0.3 `useRouter` で画面遷移

ボタンを押した後にプログラムの別ページに遷移したいときは `useRouter` を使います。`<Link>` は「ユーザーがクリックして遷移する」ためのもの、`useRouter` は「コードが勝手に遷移させる」ためのもの、と覚えると分かりやすいです。

▼ このコードがやること（先に日本語で）：ボタンを押したら、JavaScript のコードから別ページ（ `/books` ）へ自動で移動させる例です。`useRouter` で取得した `router` オブジェクトの `push` を呼ぶと、リンクをクリックしたのと同じように画面が切り替わります。初心者は「リンクは人がクリック、`router.push` はプログラムが移動させる」という違いだけ覚えておけば十分です。各メソッドの使い分けはこのあとの表とコメントで補足します。

```
// useRouter はクライアントコンポーネント専用なので、この宣言が必須
"use client";
// App Router 用の useRouter は next/navigation から取る (旧 next/router ではない)
import { useRouter } from "next/navigation";

export default function MyComponent() {
  // ルーター（ページ遷移を司るオブジェクト）を取得
  const router = useRouter();

  // ボタンクリック時に呼ばれる関数
  const handleClick = () => {
    // /books に遷移する（履歴に積まれる=戻るボタンで戻れる）
    router.push("/books");
    // 現在ページのサーバー側データを最新化（必要時のみ呼ぶ）
    router.refresh();
  };

  // クリックで handleClick を実行
  return <button onClick={handleClick}>一覧へ戻る</button>;
}
```

▼ よく使うメソッド:

メソッド	用途	例
<code>router.push("/books")</code>	新しいURLに遷移	削除後に一覧へ戻る
<code>router.replace("/login")</code>	履歴を残さず遷移	ログイン誘導
<code>router.back()</code>	ブラウザの戻るボタンと同じ	キャンセル
<code>router.refresh()</code>	現在ページを再取得	データ更新後の反映

0.4 Server Component と Client Component の使い分け（おさらい）

種類	デフォルト/宣言	できること	できないこと
Server Component	デフォルト（何も書かない）	DBに直接アクセス、 <code>await</code> で取得	<code>useState</code> / イベントハンドラ
Client Component	先頭に <code>"use client";</code> を書く	<code>useState</code> 、ボタンの <code>onClick</code> など	DB直接アクセス（API経由になる）

この章での使い分け: - 詳細ページ（読み込みのみ） → Server Component - 編集フォーム（入力に反応させる） → Client Component - 削除ボタン（確認ダイアログ + APIコール） → Client Component

フォームの送信フローについて補足: 一般的に Web のフォームは「ユーザーが入力 → 送信ボタン → ブラウザがサーバーに POST → サーバーが処理 → 結果ページを返す」という流れで動きます。この章のフォームはちょっと違って、「ユーザーが入力 → 送信ボタン → JavaScript が `e.preventDefault()` で標準送信をキャンセル → Supabase クライアント経由で直接 DB に書き込み → `router.push` でページ遷移」という SPA 的な流れになっています。標準送信ではないので、ページがリロードされず、画面のチラつきがありません。

ブラウザのキャッシュと再送信について: フォーム送信後にブラウザの「更新」ボタンを押すと、「フォームを再送信しますか？」というダイアログが出ることがあります。これは「同じ POST をもう一度送ろうとしている」状態で、二重登録の原因になります。本章のように `router.push` で別ページに遷移すると、再読み込みしても遷移先のページが更新されるだけで、フォームは再送信されません（これを Post/Redirect/Get パターンと呼びます）。

1. 詳細表示機能

書籍の詳細情報を表示するページを作成します。ここでは Next.js の動的ルーティング（Dynamic Routes）を活用します。

1-1. 動的ルーティングとは

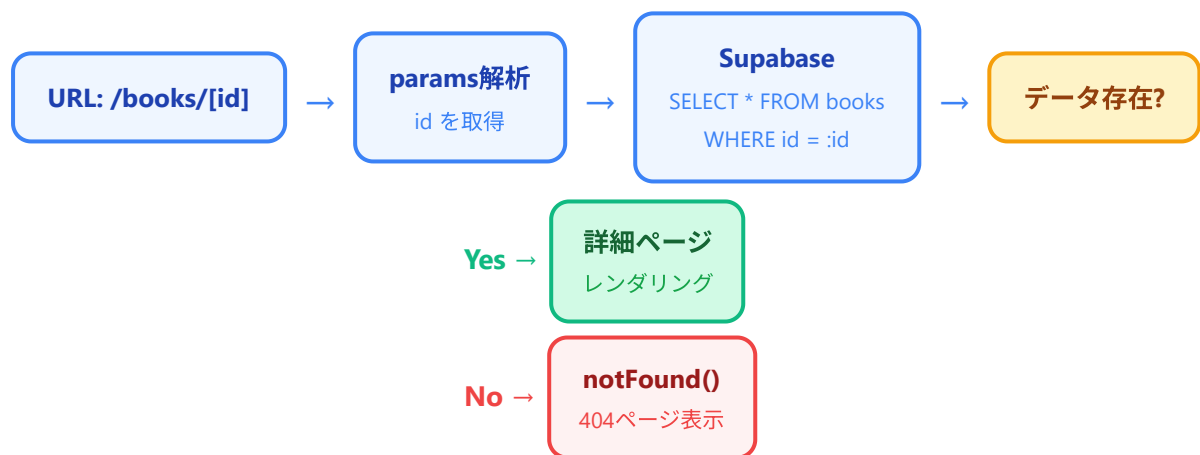
Next.js App Router では、フォルダ名を **[パラメータ名]** のように角括弧で囲むことで、動的な URL セグメントを作成できます。例えば `app/books/[id]/page.tsx` というファイルを作ると、`/books/abc123` や `/books/xyz789` のような URL にマッチし、`abc123` や `xyz789` の部分を `params.id` として取得できます。

```
app/
  books/
    page.tsx      → /books (一覧ページ)
    new/
      page.tsx    → /books/new (新規登録ページ)
    [id]/
      page.tsx    → /books/:id (詳細ページ)
      edit/
        page.tsx  → /books/:id/edit (編集ページ)
```

角括弧フォルダの優先順位: `/books/new` と `/books/[id]` の両方が存在する場合、Next.js は「具体的なパス (`new`) 」を「動的パス (`[id]`) 」より優先します。なので `/books/new` にアクセスすると `new/page.tsx` が、`/books/abc123` にアクセスすると `[id]/page.tsx` が呼ばれます。

1-2. 処理フローの概要

以下のフロー図は、詳細ページにアクセスしたときの処理の流れを示しています。



URLにアクセスすると、Next.js が URL 内の `[id]` 部分を解析して `params` オブジェクトに格納します。そのIDを使って Supabase からデータを取得し、データが存在すれば詳細ページをレンダリング、存在しなければ 404 ページを表示します。

HTTP のステータスコード: ブラウザとサーバーのやりとりには結果を表す数字 (ステータスコード) が付きます。よく見るものは `200` (正常)、`401` (認証されていない = ログインしていない)、`403` (権限がない = ログインはしているが見る資格がない)、`404` (見つからない)、`500` (サーバー側のエラー) です。本章では「存在しない ID にアクセスされたら 404」「ログイン制御を入れるなら 401/403」と覚えておけば十分です。

1-3. 書籍詳細ページの作成

まず、`app/books/[id]/` ディレクトリを作成し、その中に `page.tsx` を作成します。

ファイル: `app/books/[id]/page.tsx`

▼ このコードがやること（先に日本語で）：URL の `/books/[id]` の `[id]` 部分を手がかりに、Supabase から書籍を1件だけ取ってきて、その全情報をカード形式で表示するページです。このページはサーバー側で動く Server Component なので、`await` を使ってデータベースから直接データを読み込みます（ブラウザに認証情報を渡さずに済みます）。初心者はまず「URL から ID を取り出す → DB から1件取得 → 見つからなければ404、見つければ画面に描く」という大きな流れだけ追ってください。星表示やステータスの色分けなどの細かい処理はコード内のコメントで説明しています。

```

// サーバー側で動く Supabase クライアントを作る関数を取り込む (@/ はプロジェクトルートを指すエイリアス)
import { createClient } from "@lib/supabase/server";
// 404 ページを表示するための Next.js 組み込み関数
import { notFound } from "next/navigation";
// クライアントサイド遷移ができる Next.js のリンクコンポーネント (<a> よりも高速)
import Link from "next/link";
// この後作る削除ボタン (クライアントコンポーネント)
import DeleteButton from "@components/DeleteButton";

// -----
// 型定義
// -----
// Next.js App Router のページコンポーネントは、
// params プロパティを受け取ります。
// [id] フォルダに配置されているので、params.id で
// URLの動的部分を取得できます。
// Next.js 15 以降は params が Promise でラップ
// されている点に注意してください。
// -----
type Props = {
  // params は「{ id: string } をいずれ返す Promise」型
  params: Promise<{ id: string }>;
};

// -----
// ステータス表示用のヘルパー関数
// -----
// データベースに保存されている英語のステータス値を
// 日本語の表示ラベルに変換します。
// 例: "unread" → "未読"
// -----
// 引数 status は文字列、戻り値も文字列
function getStatusLabel(status: string): string {
  // Record<キーの型, 値の型> は「キーと値の型を指定したオブジェクト」を表す型
  const statusMap: Record<string, string> = {
    // DB の値 "unread" を「未読」に対応付け
    unread: "未読",
    // DB の値 "reading" を「読書中」に対応付け
    reading: "読書中",
    // DB の値 "finished" を「読了」に対応付け
    finished: "読了",
  };
  // 該当があれば日本語、無ければ元の値をそのまま返す (フォールバック)
  return statusMap[status] || status;
}

// -----
// ステータスに応じたバッジの色を返すヘルパー関数

```



```

// -----
// ステータスごとに異なる色のバッジを表示することで、
// ひと目で書籍の読書状態がわかるようにします。
// Tailwind CSS のクラス名を文字列として返します。
// -----
function getStatusColor(status: string): string {
  // status の値で分岐
  switch (status) {
    // 未読のとき
    case "unread":
      // 背景はグレー、文字は濃いグレー
      return "bg-gray-100 text-gray-800";
    // 読書中のとき
    case "reading":
      // 背景は薄い青、文字は濃い青
      return "bg-blue-100 text-blue-800";
    // 読了のとき
    case "finished":
      // 背景は薄い緑、文字は濃い緑
      return "bg-green-100 text-green-800";
    // 想定外の値が来たとき
    default:
      // フォールバックとしてグレー
      return "bg-gray-100 text-gray-800";
  }
}

// -----
// 評価を星マークで表示するヘルパー関数
// -----
// 数値の評価 (1~5) を視覚的な星マーク (★☆) に
// 変換して表示します。
// 例: 3 → "★★★☆☆"
// -----
// rating は数値または null (評価なし)
function renderStars(rating: number | null): string {
  // null/undefined のときは「未評価」と表示
  if (rating === null || rating === undefined) return "未評価";
  // 評価の数だけ「★」を繰り返す (例: 3 なら "★★★")
  const filled = "★".repeat(rating);
  // 残りの数だけ「☆」を繰り返す (例: 3 なら "☆☆")
  const empty = "☆".repeat(5 - rating);
  // 「★★★」+「☆☆」=「★★★☆☆」を返す
  return filled + empty;
}

// -----
// 書籍詳細ページコンポーネント (Server Component)
// -----
// このコンポーネントはサーバーサイドで実行されます。

```

```

// async 関数として定義することで、コンポーネント内で
// 直接 Supabase へのデータ取得を行えます。
// "use client" を書いていない=Server Component です。
// -----
// 分割代入で props から params を取り出し
export default async function BookDetailPage({ params }: Props) {
  // -----
  // 1. params から書籍IDを取得
  // -----
  // Next.js 15 以降、params は Promise として渡される
  // ため、await で解決する必要があります。
  // ここで await を忘れると id が undefined になり、
  // データ取得が失敗します。
  // -----
  // Promise を解決して { id } を取り出す
  const { id } = await params;

  // -----
  // 2. Supabase クライアントの作成とデータ取得
  // -----
  // サーバーコンポーネント用の Supabase クライアントを
  // 作成し、指定されたIDの書籍データを取得します。
  // .single() を使うことで、配列ではなく単一の
  // オブジェクトとしてデータを受け取ります。
  // -----
  // Cookie 連携などの非同期処理があるため await が必要
  const supabase = await createClient();

  // data を book という名前にリネームして受け取る（分割代入のリネーム構文）
  const { data: book, error } = await supabase
    // 操作対象は books テーブル
    .from("books")
    // 全カラムを取得 ("id, title" のようにカラム名を絞ることも可能)
    .select("*")
    // WHERE id = :id 条件 (URL から取った id と一致するレコード)
    .eq("id", id)
    // 結果を配列ではなく単一オブジェクトとして受け取る（0件や2件以上ならエラー）
    .single();

  // -----
  // 3. エラーハンドリング
  // -----
  // データが取得できなかった場合（IDが存在しない、
  // ネットワークエラーなど）、Next.js の notFound()
  // 関数を呼び出して 404 ページを表示します。
  //
  // notFound() は例外をスローする関数なので、
  // この行以降のコードは実行されません。
  // try/catch で囲うとエラー扱いになってしまうので、
  // 直接呼び出すのがポイントです。

```

```

// -----
// エラーがある、または book が空のとき
if (error || !book) {
  // 404 ページに即遷移（以降のコードは実行されない）
  notFound();
}

// -----
// 4. 日付のフォーマット
// -----
// データベースに保存されている日付文字列を
// 日本語の表示形式に変換します。
// 例: "2024-01-15" → "2024年1月15日"
// -----
// アロー関数で日付フォーマッタを定義
const formatDate = (dateString: string | null): string => {
  // null や空文字なら「未設定」を返す
  if (!dateString) return "未設定";
  // 文字列から Date オブジェクトを生成
  const date = new Date(dateString);
  // 日本ロケールで整形 ("ja-JP" は日本語表記)
  return date.toLocaleDateString("ja-JP", {
    // 年は数字
    year: "numeric",
    // 月は「1月」のような長い形式
    month: "long",
    // 日は数字
    day: "numeric",
  });
};

// -----
// 5. ページのレンダリング
// -----
// 詳細ページは以下のようなレイアウトで表示されます:
//
// カード形式のレイアウトで、書籍の全情報を
// 見やすく表示します。
// -----
return (
  { /* 最大幅 2xl、左右中央寄せ、上下に余白 */}
  <div className="max-w-2xl mx-auto py-8 px-4">
    { /* -----
      戻るリンク
      一覧ページに戻るためのリンクです。
      ページ上部に配置して、ユーザーがすぐに
      一覧に戻れるようにします。
      ----- */}

    <Link
      // 遷移先の URL

```

```

href="/books"
// インラインフレックスで矢印とテキストを横並びに
className="inline-flex items-center text-sm text-blue-600 hover:text-blue-800
mb-6"
>
<svg
  // 16x16 のサイズ、右に少しマージン
  className="w-4 h-4 mr-1"
  // 塗りつぶしなし
  fill="none"
  // 線の色は親要素の文字色を引き継ぐ
  stroke="currentColor"
  // SVG の表示領域
  viewBox="0 0 24 24"
>
  <path
    // 線の端を丸く
    strokeLinecap="round"
    // 線同士の結合点も丸く
    strokeLinejoin="round"
    // 線の太さ
    strokeWidth={2}
    // ← の形を描画するパス
    d="M15 19l-7-7 7-7"
  />
</svg>
一覧に戻る
</Link>

{ /* -----
  書籍詳細カード
  白背景のカードに影をつけて、情報を
  グループ化して表示します。
  ----- */}

{ /* 白背景、強めの影、角丸、はみ出し非表示 */}
<div className="bg-white shadow-lg rounded-lg overflow-hidden">
  { /* -----
    ヘッダー部分
    書籍タイトルとステータスバッジを
    表示します。背景色をグラデーションに
    して視覚的に目立たせます。
    ----- */}

  { /* 左から右へ青のグラデーション */}
  <div className="bg-gradient-to-r from-blue-600 to-blue-800 px-6 py-8">
    { /* タイトルとバッジを左右に振り分ける */}
    <div className="flex items-start justify-between">
      { /* book.title を白文字で大きく表示 */}
      <h1 className="text-2xl font-bold text-white">{book.title}</h1>
      <span
        // テンプレートリテラルで動的にクラスを合成

```

```

        className={`inline-flex items-center px-3 py-1 rounded-full text-sm font-
medium ${getStatusColor(book.status)}`}
      >
        { /* 日本語ラベルを表示 */ }
        { getStatusLabel(book.status) }
      </span>
    </div>
    /* book.author が真値（空でない）のときだけ描画する短絡評価 */
    { book.author && (
      <p className="mt-2 text-blue-100 text-lg">{book.author}</p>
    ) }
  </div>

  { /* -----
  詳細情報部分
  書籍の各項目を定義リスト風のレイアウトで
  表示します。ラベルと値を横並びにして、
  見やすく整理します。
  ----- */ }

  <div className="px-6 py-6">
    { /* dl は定義リスト。子要素間に区切り線を入れる */ }
    <dl className="divide-y divide-gray-200">
      { /* 出版社 */ }
      { /* sm 以上で 3 カラムグリッド */ }
      <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
        { /* dt は定義の見出し（ラベル） */ }
        <dt className="text-sm font-medium text-gray-500">出版社</dt>
        { /* dd は定義の本体（値）。2カラム分使う */ }
        <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
          { /* publisher が空なら「未設定」を表示 */ }
          { book.publisher || "未設定" }
        </dd>
      </div>
    </div>

    { /* 出版日 */ }
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
      <dt className="text-sm font-medium text-gray-500">出版日</dt>
      <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
        { /* 上で定義したフォーマットで整形 */ }
        { formatDate(book.published_date) }
      </dd>
    </div>

    { /* 評価 */ }
    <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
      <dt className="text-sm font-medium text-gray-500">評価</dt>
      <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
        { /* 星は黄色、少し大きめ */ }
        <span className="text-yellow-500 text-lg">
          { /* 数値を ★★☆☆☆ 形式に変換 */ }

```

```

        {renderStars(book.rating)}
      </span>
    </dd>
  </div>

  {/* ステータス */}
  <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
    <dt className="text-sm font-medium text-gray-500">ステータス</dt>
    <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
      <span
        className={`inline-flex items-center px-2.5 py-0.5 rounded-full text-
xs font-medium ${getStatusColor(book.status)}`}
      >
        {getStatusLabel(book.status)}
      </span>
    </dd>
  </div>

  {/* メモ */}
  <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
    <dt className="text-sm font-medium text-gray-500">メモ</dt>
    <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
      /* メモがあれば本文を、なければグレーで「メモはありません」を表示する三項演算子 */
      {book.memo ? (
        /* whitespace-pre-wrap で改行を保持 */
        <div className="bg-gray-50 rounded-md p-4 whitespace-pre-wrap">
          {book.memo}
        </div>
      ) : (
        <span className="text-gray-400">メモはありません</span>
      )}
    </dd>
  </div>

  {/* 登録日時 */}
  <div className="py-4 sm:grid sm:grid-cols-3 sm:gap-4">
    <dt className="text-sm font-medium text-gray-500">登録日時</dt>
    <dd className="mt-1 text-sm text-gray-900 sm:col-span-2 sm:mt-0">
      /* DB が自動で入れる作成日時 */
      {formatDate(book.created_at)}
    </dd>
  </div>
</dl>
</div>

{/* -----
アクションボタン部分
編集と削除の操作ボタンを表示します。
編集はリンク（青色）、削除はボタン
（赤色）で表示して、操作を視覚的に

```

区別します。

```
----- */}

{/* 右寄せでボタンを横並び、要素間隔 3 */}
<div className="bg-gray-50 px-6 py-4 flex items-center justify-end space-x-3">
  <Link
    // テンプレートリテラルで動的に編集ページの URL を組み立て
    href={`\`/books/${book.id}/edit`\`}
    className="inline-flex items-center px-4 py-2 border border-transparent
text-sm font-medium rounded-md shadow-sm text-white bg-blue-600 hover:bg-blue-700
focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-blue-500"
  >
    <svg
      className="w-4 h-4 mr-2"
      fill="none"
      stroke="currentColor"
      viewBox="0 0 24 24"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        strokeWidth={2}
        // 鉛筆（編集）アイコンのパス
        d="M11 5H6a2 2 0 0-2 2v11a2 2 0 02 2h11a2 2 0 02-2v-5m-1.414-9.414a2
2 0 112.828 2.828L11.828 15H9v-2.828l8.586-8.586z"
      />
    </svg>
    編集する
  </Link>

  {/* -----
DeleteButton はクライアントコンポーネント
です。window.confirm による確認ダイアログ
やクリックイベントの処理はブラウザ側で
行う必要があるためです。
サーバーコンポーネントから props を渡せる
点に注目 (id と title を渡している)。
----- */}

  <DeleteButton bookId={book.id} bookTitle={book.title} />
</div>
</div>
</div>
);
}
```

このコードのポイント:

- `params` は Next.js 15 以降で `Promise` として渡されるため、`await params` で値を取得します。古いバージョンの Next.js では直接 `params.id` でアクセスできましたが、最新版ではこの非同期パターンが必要です。`await` を忘れると `id` が `undefined` になり、Supabase のクエリが失敗します。

- `notFound()` は `next/navigation` から import する関数で、呼び出すと即座に 404 ページが表示されます。内部的には例外をスローするため、`notFound()` 以降のコードは実行されません。ファイル単位で `not-found.tsx` を用意するとカスタマイズもできます。
- `.single()` メソッドは、クエリ結果が正確に1件であることを期待します。0件や2件以上の場合はエラーになります。「存在しないかも」を許容したいときは `.maybeSingle()` (0件のときは `data: null`、エラーにしない) を使うこともあります。
- サーバーコンポーネントなので `"use client"` ディレクティブは不要です。データ取得はすべてサーバーサイドで行われ、完成した HTML がクライアントに送信されます。ブラウザ側に Supabase の認証情報を晒さなくて済むメリットもあります。

詳細ページは上記のコード内のコメントで示した通り、カード形式のレイアウトで表示されます。完成イメージは以下の通りです:

[← 一覧に戻る](#)

書籍タイトル

著者	山田太郎
出版社	技術評論社
出版日	2024年1月15日
評価	★★★★☆
ステータス	読書中

メモ

とても良い本です。特に第3章が参考になりました。実務でも活用できる内容が多いです。

編集する

削除する

ページ上部にはグラデーション背景のヘッダーがあり、書籍タイトル、著者名、ステータスバッジが表示されます。その下に出版社、出版日、評価（星マーク）、ステータス、メモ、登録日時が定義リスト風に並びます。ページ最下部にはグレー背景のフッターに「編集する」ボタン（青色）と「削除する」ボタン（赤色）が配置されます。

2. 編集機能

書籍情報を編集する機能を実装します。編集機能では、前章で作成した **BookForm** コンポーネントを再利用し、既存データを初期値としてフォームに表示します。

「編集」と冪等性: 編集（UPDATE）は何度実行しても最終結果が同じになる操作（冪等性がある）です。同じデータで2回更新ボタンを押しても、データベースの状態は同じになります。これは新規登録（INSERT）と大きく異なる点で、INSERT は押すたびにレコードが増えてしまいます。

同時編集の競合について: 複数のユーザーが同じ書籍を同時に編集して保存すると、後から保存した方の内容で上書きされます（最後の書き込みが勝つ＝**Last Write Wins**）。このチュートリアルの実装ではこの問題を意識せず単純に上書きしますが、より高度なアプリでは「更新日時を比較して古いデータの上書きを拒否する」「ロックを取る」といった対策が必要になります。

2-1. 編集処理のフロー

Phase 1: データ読み込み

- 1 ユーザー → ブラウザ: `/books/[id]/edit` にアクセス
- 2 編集ページ → Supabase: `SELECT * FROM books WHERE id = :id`
- 3 Supabase → 編集ページ → ブラウザ: フォーム（既存データ入り）をレンダリング

--- ユーザーがフォームを編集 ---

Phase 2: データ更新

- 4 ユーザー: 「更新」 ボタンをクリック
- 5 ブラウザ → Supabase: `UPDATE books SET ... WHERE id = :id`
- 6 更新完了: `router.push("/books/[id]")` → 詳細ページにリダイレクト

このフロー図が示すように、編集機能は2段階の処理で構成されています。まずページアクセス時にサーバーサイドで既存データを取得し、フォームの初期値として設定します（これを **編集前データのプリロード** と呼びます）。次にユーザーがフォームを編集して「更新」ボタンを押すと、クライアントサイドで Supabase の UPDATE クエリが実行され、完了後に詳細ページへリダイレクトされます。

楽観的 UI (Optimistic UI) について: より高速な UX を実現する手法として、サーバーの応答を待たずに「成功したであろう状態」を画面に先に反映してしまう手法があります。これを楽観的 UI と呼びます。本章の実装は逆に「サーバーの応答を待ってから反映する」シンプルな方式（悲観的更新）です。初学者にはこちらが分かりやすいので、まずは応答を待つパターンで作ります。

2-2. BookForm の更新（編集対応）

前章で作成した `BookForm` コンポーネントを、新規登録と編集の両方に対応できるように更新します。主な変更点は以下の通りです。

- `initialData` prop: 編集時に既存データをフォームの初期値として渡す
- `isEdit` prop: 新規登録か編集かを区別する
- ボタンテキスト: 新規登録時は「登録する」、編集時は「更新する」
- 送信先の処理: 新規登録時は INSERT、編集時は UPDATE

defaultValue と value の違い: React の input には 2 通りの値の渡し方があります。 - `defaultValue` : 最初に 1 回だけ反映される（非制御コンポーネント）。ユーザーの入力は React の状態に同期されない。 - `value` : 常に渡された値が表示される（制御コンポーネント）。`onChange` で React の状態を更新しないと、入力できなくなる。本章のフォームは `value` + `onChange` の制御コンポーネント方式です。状態を React 側で完全に管理するので、検証や送信処理が書きやすくなります。

ファイル: `components/BookForm.tsx`

▼ このコードがやること（先に日本語で）: 書籍の入力フォームを、「新規登録」と「編集」の両方で使い回せるように作り直します。ポイントは props で渡される `isEdit` というフラグで、これが `true` のときは Supabase の UPDATE（更新）を、`false` のときは INSERT（新規追加）を実行するよう処理を切り替えます。編集時は `initialData` で渡された既存データをフォームの初期値に入れておくので、最初から内容が埋まった状態で表示されます。初心者はまず「1つのフォームが2役をこなしている」「入力値は `useState` で React が管理している」という2点を押さえてください。各フィールドや送信処理の詳細はコード内のコメントにあります。

```

// このファイルはブラウザ側で動くクライアントコンポーネント。useState/useRouter を使うために必須
"use client";

// 状態管理フック
import { useState } from "react";
// ページ遷移フック (App Router 用)
import { useRouter } from "next/navigation";
// ブラウザ向けの Supabase クライアント
import { createClient } from "@lib/supabase/client";

// -----
// 型定義
// -----
// BookFormData: フォームで扱うデータの型
// initialData に渡す型でもあり、フォームの状態
// 管理にも使用します。
// 数値は null も許容 (評価が未入力の場合用)。
// -----
type BookFormData = {
  // タイトル (必須)
  title: string;
  // 著者
  author: string;
  // 出版社
  publisher: string;
  // 出版日 (YYYY-MM-DD 形式の文字列)
  published_date: string;
  // 評価 (1~5 または null)
  rating: number | null;
  // ステータス ("unread" / "reading" / "finished")
  status: string;
  // メモ
  memo: string;
};

// -----
// Props の型定義
// -----
// initialData: 編集時に既存のデータを渡すための
//               プロパティ。新規登録時は undefined。
// isEdit:       編集モードかどうかを示すフラグ。
//               true の場合は UPDATE、false の場合は
//               INSERT を実行します。
// bookId:       編集時に対象の書籍IDを渡すための
//               プロパティ。UPDATE の WHERE 句で使用。
// 末尾の ? は「省略可能」を表す TypeScript の記法です。
// -----
type Props = {
  // 省略可 (新規登録時は渡さない)

```

```

initialData?: BookFormData;
// 省略可 (デフォルト false)
isEdit?: boolean;
// 省略可 (編集時のみ必要)
bookId?: string;
};

// -----
// デフォルトの初期値
// -----
// 新規登録時に使用するフォームの初期値です。
// すべてのフィールドを空またはデフォルト値で
// 初期化します。
// -----
const defaultFormData: BookFormData = {
  // 空文字
  title: "",
  // 空文字
  author: "",
  // 空文字
  publisher: "",
  // 空文字 (date 入力欄の空状態)
  published_date: "",
  // 評価なし
  rating: null,
  // デフォルトは「未読」
  status: "unread",
  // 空文字
  memo: "",
};

export default function BookForm({
  // 編集時の初期データ
  initialData,
  // デフォルト値を指定 (渡されなければ false)
  isEdit = false,
  // 編集対象の ID
  bookId,
}: Props) {
  // 遷移用のルーターを取得
  const router = useRouter();

  // -----
  // フォームの状態管理
  // -----
  // initialData が渡された場合 (編集モード) は
  // 既存データを初期値として使用します。
  // 渡されなかった場合 (新規登録モード) は
  // defaultFormData を初期値として使用します。
  // ?? は「左が null/undefined なら右を使う」演算子 (nullish coalescing) です。

```

```

// -----
const [formData, setFormData] = useState<BookFormData>(
  // initialData が undefined なら defaultFormData
  initialData ?? defaultFormData
);
// 送信中フラグ (二重送信防止)
const [isSubmitting, setIsSubmitting] = useState(false);
// エラーメッセージ (無ければ null)
const [error, setError] = useState<string | null>(null);

// -----
// フォームフィールドの値変更ハンドラ
// -----
// 各入力フィールドの onChange イベントで呼ばれ、
// フォームの状態を更新します。
// name 属性をキーとして、対応するフィールドの
// 値だけを更新します。
// -----
const handleChange = (
  // ChangeEvent は React の入力変化イベントの型
  e: React.ChangeEvent<
    // input / select / textarea のいずれかに対応
    HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement
  >
) => {
  // イベントの発生元から name と value を取り出す
  const { name, value } = e.target;
  // 関数形式の setState (前の状態を引数に取れる)
  setFormData((prev) => ({
    // スプレッド構文で既存の値を全部展開 (他のフィールドはそのまま)
    ...prev,
    // 計算プロパティ名で name に対応するフィールドだけ上書き
    [name]: value,
  }));
};

// -----
// 評価の変更ハンドラ
// -----
// 評価は数値として扱うため、専用のハンドラを
// 用意します。空文字の場合は null を設定し、
// それ以外は数値に変換します。
// select の value は常に文字列で来るので、明示的に
// parseInt で数値化する必要があります。
// -----
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  // 選択された値 (文字列)
  const value = e.target.value;
  setFormData((prev) => ({
    ...prev,

```

```

    // 空なら null、そうでなければ 10 進数で整数化
    rating: value === "" ? null : parseInt(value, 10),
  });
};

// -----
// フォーム送信ハンドラ
// -----
// isEdit フラグに応じて INSERT または UPDATE を
// 実行します。
// async 関数として定義し、await で非同期処理の
// 完了を待ちます。
// -----
const handleSubmit = async (e: React.FormEvent) => {
  // フォームのデフォルト送信（ページリロード）をキャンセル
  e.preventDefault();
  // 送信中状態に
  setIsSubmitting(true);
  // 過去のエラーをクリア
  setError(null);

  // -----
  // バリデーション
  // -----
  // タイトルは必須項目です。空の場合はエラーを
  // 表示して処理を中断します。
  // trim() で前後の空白を除去してからチェックする
  // ことで「スペースだけ」の入力もエラー扱いに。
  // -----
  // タイトルが空白のみ または 空文字の場合
  if (!formData.title.trim()) {
    setError("タイトルは必須です。");
    // 送信中フラグを戻す
    setIsSubmitting(false);
    // 処理を中断
    return;
  }

  try {
    // クライアント用 Supabase インスタンスを作成
    const supabase = createClient();

    // 編集モード かつ ID がある場合
    if (isEdit && bookId) {
      // -----
      // 編集モード: UPDATE
      // -----
      // 既存の書籍データを更新します。
      // .eq("id", bookId) で対象のレコードを指定し、
      // フォームの内容で上書きします。

```

```

// .eq() を省略すると全件更新されてしまうので
// 絶対に忘れないこと。
// -----
const { error: updateError } = await supabase
  // books テーブルに対して
  .from("books")
  // UPDATE を発行
  .update({
    // 前後の空白を除去
    title: formData.title.trim(),
    // 空文字なら null (DB をクリーンに保つ)
    author: formData.author.trim() || null,
    publisher: formData.publisher.trim() || null,
    // 空文字なら null
    published_date: formData.published_date || null,
    // null または数値
    rating: formData.rating,
    status: formData.status,
    memo: formData.memo.trim() || null,
  })
  // WHERE id = bookId
  .eq("id", bookId);

// UPDATE 中にエラーが起きたら
if (updateError) {
  // catch ブロックに飛ばす
  throw updateError;
}

// -----
// 更新成功: 詳細ページにリダイレクト
// -----
// 更新が完了したら、その書籍の詳細ページに
// 遷移します。ユーザーは更新後の情報を
// すぐに確認できます。
// refresh() でサーバーコンポーネントのキャッシュ
// も更新し、最新のデータが表示されるようにします。
// -----
// 詳細ページに遷移
router.push(`/books/${bookId}`);
// サーバー側データを再取得
router.refresh();
} else {
  // -----
  // 新規登録モード: INSERT
  // -----
  // 新しい書籍データを挿入します。
  // .select() を付けることで、挿入されたデータ
  // (自動生成されたIDを含む) を取得できます。
  // -----

```

```

const { data, error: insertError } = await supabase
  .from("books")
  // INSERT 文に相当
  .insert({
    title: formData.title.trim(),
    author: formData.author.trim() || null,
    publisher: formData.publisher.trim() || null,
    published_date: formData.published_date || null,
    rating: formData.rating,
    status: formData.status,
    memo: formData.memo.trim() || null,
  })
  // 挿入したレコードを返してもらう
  .select()
  // 1 件だけ取得
  .single();

if (insertError) {
  throw insertError;
}

// -----
// 登録成功: 一覧ページにリダイレクト
// -----
router.push("/books");
router.refresh();
}
} catch (err) {
  // 開発者向けにコンソールへエラー出力
  console.error("保存エラー:", err);
  setError(
    isEdit
      // 編集時のメッセージ
      ? "書籍の更新に失敗しました。もう一度お試しください。"
      // 新規登録時のメッセージ
      : "書籍の登録に失敗しました。もう一度お試しください。"
  );
} finally {
  // 成功・失敗どちらでも、送信中フラグを戻す (finally で必ず実行)
  setIsSubmitting(false);
}
};

// -----
// フォームのレンダリング
// -----
return (
  /* form の onSubmit で送信時に handleSubmit が呼ばれる */
  <form onSubmit={handleSubmit} className="space-y-6">
    /* エラーメッセージ */

```



```

/* error が null でないときだけ描画 */
{error && (
  /* 赤系の警告ボックス */
  <div className="bg-red-50 border-l-4 border-red-400 p-4 rounded">
    <div className="flex">
      <div className="flex-shrink-0">
        <svg
          className="h-5 w-5 text-red-400"
          viewBox="0 0 20 20"
          fill="currentColor"
        >
          <path
            fillRule="evenodd"
            // x印アイコンのパス
            d="M10 18a8 8 0 10-16 8 8 0 00 16 8z"
            clipRule="evenodd"
          />
        </svg>
      </div>
      <div className="ml-3">
        /* エラー文言を表示 */
        <p className="text-sm text-red-700">{error}</p>
      </div>
    </div>
  </div>
)}

/* タイトル (必須) */
<div>
  <label
    // この label がどの input と紐づくか (id と一致させる)
    htmlFor="title"
    className="block text-sm font-medium text-gray-700"
  >
    /* 赤いアスタリスクで必須を示す */
    タイトル <span className="text-red-500">*</span>
  </label>
  <input
    // テキスト入力欄
    type="text"
    // label の htmlFor と一致
    id="title"
    // handleChange で参照される名前
    name="title"
    // HTML 側の必須チェック (ブラウザがエラー表示)
    required
    // 制御コンポーネント方式: state の値を反映
    value={formData.title}
  >

```

```

        // 入力のたびに state を更新
        onChange={handleChange}
        className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
        // 未入力時のヒント
        placeholder="書籍のタイトルを入力"
      />
    </div>

    {/* 著者 */}
    <div>
      <label
        htmlFor="author"
        className="block text-sm font-medium text-gray-700"
      >
        著者
      </label>
      <input
        type="text"
        id="author"
        // name="author" → handleChange で formData.author が更新される
        name="author"
        value={formData.author}
        onChange={handleChange}
        className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
        placeholder="著者名を入力"
      />
    </div>

    {/* 出版社 */}
    <div>
      <label
        htmlFor="publisher"
        className="block text-sm font-medium text-gray-700"
      >
        出版社
      </label>
      <input
        type="text"
        id="publisher"
        name="publisher"
        value={formData.publisher}
        onChange={handleChange}
        className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
        placeholder="出版社名を入力"
      />
    </div>

```

```

{ /* 出版日 */ }
<div>
  <label
    htmlFor="published_date"
    className="block text-sm font-medium text-gray-700"
  >
    出版日
  </label>
  <input
    // 日付ピッカー (YYYY-MM-DD 形式の文字列を扱う)
    type="date"
    id="published_date"
    name="published_date"
    value={formData.published_date}
    onChange={handleChange}
    className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
  />
</div>

{ /* 評価 */ }
<div>
  <label
    htmlFor="rating"
    className="block text-sm font-medium text-gray-700"
  >
    評価
  </label>
  <select
    id="rating"
    name="rating"
    // null のとき "" を渡す (select は string しか扱えない)
    value={formData.rating ?? ""}
    // 専用ハンドラで数値化
    onChange={handleRatingChange}
    className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
  >
    { /* 空文字を選択肢として用意 */ }
    <option value="">未評価</option>
    <option value="1">★☆☆☆☆ (1) </option>
    <option value="2">★★☆☆☆ (2) </option>
    <option value="3">★★★☆☆ (3) </option>
    <option value="4">★★★★☆ (4) </option>
    <option value="5">★★★★★ (5) </option>
  </select>
</div>

{ /* ステータス */ }
<div>

```

```

<label
  htmlFor="status"
  className="block text-sm font-medium text-gray-700"
>
  ステータス
</label>
<select
  id="status"
  name="status"
  value={formData.status}
  // ステータスは文字列のままなので共通ハンドラで OK
  onChange={handleChange}
  className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
>
  <option value="unread">未読</option>
  <option value="reading">読書中</option>
  <option value="finished">読了</option>
</select>
</div>

{/* ✕モ */}
<div>
  <label
    htmlFor="memo"
    className="block text-sm font-medium text-gray-700"
  >
    ✕モ
  </label>
  <textarea
    id="memo"
    name="memo"
    // 表示行数の目安
    rows={4}
    value={formData.memo}
    onChange={handleChange}
    className="mt-1 block w-full rounded-md border-gray-300 shadow-sm
focus:border-blue-500 focus:ring-blue-500 sm:text-sm"
    placeholder="読書✕モや感想を入力"
  />
</div>

{/* 送信ボタン */}
{/* ボタンを右寄せで横並び */}
<div className="flex items-center justify-end space-x-3">
  <button
    // type="button" は「フォーム送信をしないボタン」
    type="button"
    // クリック時に 1 つ前のページへ戻る
    onClick={() => router.back()}
  >

```

```

        className="inline-flex items-center px-4 py-2 border border-gray-300 text-sm
font-medium rounded-md text-gray-700 bg-white hover:bg-gray-50 focus:outline-none
focus:ring-2 focus:ring-offset-2 focus:ring-blue-500"
      >
        キャンセル
      </button>
      <button
        // type="submit" でフォームの onSubmit を発火させる
        type="submit"
        // 送信中はクリック不可（二重送信防止）
        disabled={isSubmitting}
        className="inline-flex items-center px-4 py-2 border border-transparent text-
sm font-medium rounded-md shadow-sm text-white bg-blue-600 hover:bg-blue-700
focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-blue-500
disabled:opacity-50 disabled:cursor-not-allowed"
      >
        /* 送信中なら... */
        {isSubmitting
          /* かつ編集モードなら */
          ? isEdit
            /* 「更新中...」 */
            ? "更新中..."
            /* それ以外は「登録中...」 */
            : "登録中..."
          /* 送信中でなくて編集モードなら */
          : isEdit
            /* 「更新する」 */
            ? "更新する"
            /* 新規登録なら「登録する」 */
            : "登録する"}
        </button>
      </div>
    </form>
  );
}

```

BookForm の主な変更点の解説:

1. **initialData** prop: 編集時に既存のデータをフォームの初期値として渡します。**useState** の初期値で **initialData ?? defaultFormData** と記述することで、initialData が渡された場合はそのデータを、渡されなかった場合はデフォルト値を使用します。これが「編集前データのプリロード」の中核です。
2. **isEdit** prop: このフラグが **true** の場合、フォーム送信時に **INSERT** ではなく **UPDATE** を実行します。デフォルト値は **false**（新規登録モード）です。
3. **bookId** prop: 編集時に UPDATE の対象を特定するために必要です。**.eq("id", bookId)** で特定のレコードだけを更新します。これを忘れると「全件更新」という事故につながるので注意してください。

4. ボタンテキストの切り替え: `isEdit` フラグに応じて、ボタンのテキストが「登録する」と「更新する」で切り替わります。送信中の表示も「登録中...」と「更新中...」で異なります。
5. リダイレクト先の違い: 新規登録後は一覧ページ（`/books`）へ、編集後は詳細ページ（`/books/${bookId}`）へリダイレクトします。

`type="submit"` と `type="button"` の違い: `<form>` 内のボタンは、`type` 属性を指定しないと既定で `submit` 扱いになり、クリックするとフォームが送信されてしまいます。「キャンセル」のように送信を伴わないボタンには必ず `type="button"` を明示してください。逆に「更新する／登録する」ボタンは送信したいので `type="submit"` を指定し、フォームの `onSubmit` ハンドラを発火させます。

編集フォームにアクセスすると、各入力フィールドに既存のデータが入った状態で表示されます。ユーザーは変更したい項目だけを修正して「更新する」ボタンを押せば、データベースが更新されます。

▼ コードを1つずつ分解して解説

上の `BookForm` には初心者がつまづきやすい書き方がいくつもあります。順番に、塊ごとにていねいに見ていきましょう。

解説1: props で「新規」と「編集」を切り替える

```
type Props = {
  // 省略可（新規登録時は渡さない）
  initialData?: BookFormData;
  // 省略可（デフォルト false）
  isEdit?: boolean;
  // 省略可（編集時のみ必要）
  bookId?: string;
};
```

- このフォームは「新規登録」と「編集」の2役を1つで兼ねます。その切り替えに使うのが上の3つの props（親から渡される入力データ）です。
- `initialData` は「編集時にフォームへ最初から入れておく既存データ」です。新規登録のときは渡さない（＝中身が空）ので、末尾の `?` で「省略可（あってもなくてもよい）」にしています。
- `isEdit` は「今が編集モードかどうか」を表す `true` / `false` の旗（フラグ）です。これが `true` なら UPDATE（更新）、`false` なら INSERT（新規追加）を実行します。
- `bookId` は「どの書籍を更新するか」を表すIDで、編集時だけ必要になります。

用語: `prop`（プロップ）は親コンポーネントから子へ渡す入力データのこと。`オプション`（`?`）は「その項目を省略してもよい」というTypeScriptの記法で、省略されると値は `undefined` になります。

解説2: 初期値を `initialData ?? defaultFormData` で決める

```
const [formData, setFormData] = useState<BookFormData>(  
  // initialData が undefined なら defaultFormData  
  initialData ?? defaultFormData  
);
```

- `useState` (ユーズ・ステート) は「変化する値を覚えておく箱」を作る道具で、`[今の値, 値を変える関数]` の形で受け取ります。ここでは入力内容をまとめて `formData` という1つの箱に入れています。
- `initialData ?? defaultFormData` の `??` (ナリッシュ合体演算子) は「左が `null` か `undefined` のときだけ右を使う」という意味です。
- つまり「編集モードで `initialData` が渡されていればそれを初期値に、渡されていなければ(新規登録なら)空っぽの `defaultFormData` を初期値に」という切り替えを1行で実現しています。これが「編集前データのプリロード(事前読み込み)」の中核です。

用語: `??` (nullish coalescing) は「左が `null` / `undefined` のときだけ右の値を採用する」演算子。 `||` と似ていますが、 `||` は `0` や `""` も右に置き換えてしまうのに対し、 `??` は `null` / `undefined` だけを対象にする点が違います。

解説3: 1つの `handleChange` で全フィールドを更新する

```
const handleChange = (  
  e: React.ChangeEvent<  
    HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement  
  >  
) => {  
  // イベントの発生元から name と value を取り出す  
  const { name, value } = e.target;  
  // 関数形式の setState (前の状態を引数に取れる)  
  setFormData((prev) => ({  
    // スプレッド構文で既存の値を全部展開 (他のフィールドはそのまま)  
    ...prev,  
    // 計算プロパティ名で name に対応するフィールドだけ上書き  
    [name]: value,  
  }));  
};
```

- 入力欄に文字を打つたびに呼ばれる関数です。 `e.target` (変化が起きた入力欄そのもの) から、 `name` (その欄の名前) と `value` (今入力されている値) を取り出します。
- `...prev` (スプレッド構文) は「今ある全項目をまるごとコピー」する書き方です。Reactでは「古い state を直接書き換えず、新しいオブジェクトを作って差し替える」のがルールなので、こうして全項目をコピーしてから1つだけ上書きします。

- `[name]: value`（動的なキー）は「変数 `name` の中身を、そのままキー（項目名）として使う」書き方です。
`name` が `"title"` なら `title: value` と書いたのと同じになります。
- これにより、たった1つの `handleChange` で、タイトル・著者・出版社...どの欄が変わっても、その欄だけを正しく更新できます。

用語: スプレッド構文（`...`）はオブジェクトや配列の中身を展開してコピーする記法。動的なキー（`computed property name`、`[name]`）は変数の値をそのままプロパティ名として使う記法です。

解説4: `isEdit` で UPDATE と INSERT を切り替える

```
// 編集モード かつ ID がある場合
if (isEdit && bookId) {
  const { error: updateError } = await supabase
    // books テーブルに対して
    .from("books")
    // UPDATE を発行
    .update({
      // 前後の空白を除去
      title: formData.title.trim(),
      // 空文字なら null (DB をクリーンに保つ)
      author: formData.author.trim() || null,
      publisher: formData.publisher.trim() || null,
      // 空文字なら null
      published_date: formData.published_date || null,
      // null または数値
      rating: formData.rating,
      status: formData.status,
      memo: formData.memo.trim() || null,
    })
    // WHERE id = bookId
    .eq("id", bookId);
```

- 送信処理の中心です。`if (isEdit && bookId)` で「編集モードかつIDがある」ときだけ UPDATE（更新）を実行し、そうでなければ（`else` 側で）INSERT（新規追加）を実行します。
- `.update({ ... })` で更新したい内容を渡し、`.eq("id", bookId)` で「IDが一致する1件だけ」に絞り込みます。この `.eq()` を忘れると全件が更新されてしまうため、UPDATE/DELETE では絶対に付けます。
- `formData.author.trim() || null` は「前後の空白を除いた結果が空文字なら `null` を入れる」という意味です（`||` は左が空っぽのとき右を使う）。空文字ではなく `null` を入れることで、データベースを「未入力 = null」というきれいな状態に保てます。

用語: `.eq("id", bookId)` は「id 列が bookId と等しい行だけ」を対象にする絞り込み（SQL の `WHERE id = ...` に相当）。`UPDATE` は既存データの書き換え、`INSERT` は新しいデータの追加を指すデータベース操作です。

解説5: 送信中フラグでボタン文言を出し分ける

```
/* 送信中なら... */
{isSubmitting
  /* かつ編集モードなら */
  ? isEdit
    /* 「更新中...」 */
    ? "更新中..."
    /* それ以外は「登録中...」 */
    : "登録中..."
  /* 送信中でなくて編集モードなら */
  : isEdit
    /* 「更新する」 */
    ? "更新する"
    /* 新規登録なら「登録する」 */
    : "登録する"}
```

- ボタンに表示する文字を、2つの旗（`isSubmitting` と `isEdit`）の組み合わせで4通りに出し分けています。
- 三項演算子（`条件 ? A : B`）を入れ子（ネスト）にしており、まず `isSubmitting`（送信中か）で分け、その中でさらに `isEdit`（編集か）で分けています。結果は「更新中... / 登録中... / 更新する / 登録する」の4パターンです。
- このように送信中は「更新中...」等に変え、同時にボタンを `disabled`（クリック不可）にすることで、ユーザーに処理中だと伝えつつ二重送信（ボタン連打による重複処理）を防いでいます。

用語: 三項演算子（`?:`）は「条件？ 真のときの値：偽のときの値」と書く式。ネストは構文を入れ子にすること。二重送信はボタン連打などで同じ処理が複数回走ってしまう不具合のことです。

2-3. 編集ページの作成

`app/books/[id]/edit/` ディレクトリを作成し、`page.tsx` を作成します。このページはサーバーコンポーネントとして動作し、既存の書籍データを取得して `BookForm` に渡す役割を持ちます。

ファイル: `app/books/[id]/edit/page.tsx`

▼このコードがやること（先に日本語で）：編集ページの本体です。URL の `[id]` から書籍を1件取得し、その既存データを `initialData` として `BookForm` に渡すのが役割です。このページ自体はサーバー側でデータを読むだけの Server Component で、実際の入力や更新処理は受け取った側の `BookForm`（クライアント側）が担当します。初心者は「重いデータ取得はサーバーで行い、対話的なフォームはクライアントに任せる」という役割分担に注目してください。`null` を空文字に変換している理由などはコード内のコメントで説明しています。

```

// サーバー用 Supabase クライアント
import { createClient } from "@lib/supabase/server";
// 404 表示用
import { notFound } from "next/navigation";
// 戻るリンク用
import Link from "next/link";
// 上で更新した共通フォーム
import BookForm from "@components/BookForm";

// -----
// 型定義
// -----
// 動的セグメント [id] の値を Promise として受け取る
// (Next.js 15 以降の仕様)。
// -----
type Props = {
  params: Promise<{ id: string }>;
};

// -----
// 編集ページコンポーネント (Server Component)
// -----
// このコンポーネントは以下の処理を行います:
// 1. URLのパラメータから書籍IDを取得
// 2. Supabase から既存の書籍データを取得
// 3. データが存在しなければ 404 を表示
// 4. BookForm コンポーネントに既存データを渡して
// レンダリング
// -----
export default async function BookEditPage({ params }: Props) {
  // -----
  // 1. params から書籍IDを取得
  // -----
  // await を忘れると id が undefined になり、
  // データ取得時にエラーになります。
  // -----
  const { id } = await params;

  // -----
  // 2. Supabase から既存データを取得
  // -----
  // クライアントを準備
  const supabase = await createClient();

  const { data: book, error } = await supabase
    .from("books")
    .select("*")
    // URL の id と一致するレコード
    .eq("id", id)

```

```

// 単一レコードとして取得
.single();

// -----
// 3. エラーハンドリング
// -----
// 存在しない ID にアクセスされた場合や、
// 何らかのエラーが起きた場合は 404 を表示します。
// -----
if (error || !book) {
  notFound();
}

// -----
// 4. BookForm に渡す初期データを整形
// -----
// データベースから取得したデータをフォームの型に
// 合わせて整形します。null の値は空文字に変換し、
// フォームの入力フィールドで正しく表示されるように
// します。
//
// ?? は左が null/undefined のときだけ右の値を使う
// 演算子です (nullish coalescing)。
// -----
const initialData = {
  // null なら空文字に変換
  title: book.title ?? "",
  author: book.author ?? "",
  publisher: book.publisher ?? "",
  published_date: book.published_date ?? "",
  // null はそのまま
  rating: book.rating ?? null,
  // status は null だと困るのでデフォルトを設定
  status: book.status ?? "unread",
  memo: book.memo ?? "",
};

// -----
// 5. レンダリング
// -----
return (
  <div className="max-w-2xl mx-auto py-8 px-4">
    {/* 戻るリンク */}
    <Link
      // 編集元の詳細ページへ
      href={`\books/${id}`}
      className="inline-flex items-center text-sm text-blue-600 hover:text-blue-800
mb-6"
    >
      <svg

```

```

        className="w-4 h-4 mr-1"
        fill="none"
        stroke="currentColor"
        viewBox="0 0 24 24"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          strokeWidth={2}
          // 左向き矢印
          d="M15 19l-7-7 7-7"
        />
      </svg>
      詳細に戻る
    </Link>

    { /* ページタイトル */ }
    <div className="mb-8">
      <h1 className="text-2xl font-bold text-gray-900">書籍を編集</h1>
      <p className="mt-1 text-sm text-gray-600">
        { /* 既存タイトルを表示して、編集対象を明示 */ }
        「{book.title}」の情報を編集します。
      </p>
    </div>

    { /* -----
      BookForm コンポーネント
      initialData: 既存の書籍データ
      isEdit: true (編集モード)
      bookId: 対象の書籍ID
      サーバーコンポーネントからクライアント
      コンポーネントへ props を渡す例。
      ----- */ }
    <div className="bg-white shadow rounded-lg p-6">
      <BookForm initialData={initialData} isEdit={true} bookId={id} />
    </div>
  </div>
);
}

```

このコードのポイント:

- 編集ページもサーバーコンポーネントです。データ取得はサーバーサイドで行い、取得したデータを `BookForm`（クライアントコンポーネント）に props として渡します。「重い処理はサーバー」「対話的な処理はクライアント」という Next.js App Router の典型的な役割分担です。
- `initialData` の整形では、データベースの `null` 値を空文字に変換しています。これにより、フォームの入力フィールドが React の controlled component として正しく動作します（`null` を value に渡すと uncontrolled component になる）。

- 戻るリンクは詳細ページ（`/books/${id}`）を指しています。一覧ではなく、編集元の詳細ページに戻る方がユーザー体験として自然です。

Server Action という別パターン: 本章ではクライアントコンポーネント側で Supabase を直接呼び出していますが、Next.js には「Server Action」という別の更新手段もあります。「`use server`」ディレクティブを付けた関数をフォームに直接渡し、サーバー側で更新→`revalidatePath` でキャッシュ破棄→`redirect` で遷移、という流れにできます。今回の教材では学習コストを抑えるためクライアント直書きを採用していますが、認証情報を守りたい場合や複雑なバリデーションをサーバーで行いたい場合は Server Action の方が向いています。

`revalidatePath("/books")` を呼ぶと、その URL に紐づくサーバーコンポーネントのキャッシュが破棄され、次のアクセスで最新データが取得されます。

▼ コードを1つずつ分解して解説

編集ページの中でも、初心者が押さえておきたい塊を順に見ていきます。

解説1: URL の id で1件だけ取得する

```
const { id } = await params;

// クライアントを準備
const supabase = await createClient();

const { data: book, error } = await supabase
  .from("books")
  .select("*")
  // URL の id と一致するレコード
  .eq("id", id)
  // 単一レコードとして取得
  .single();
```

- このページはサーバー側で動く Server Component なので、`await` を使ってデータベースから直接データを読めます。
- `const { id } = await params;` で、URL の `[id]` 部分を取り出します。Next.js 15 以降 `params` は Promise（あとで値が決まる箱）なので、`await` を付けて中身を取り出す点が要注意です（忘れると `id` が `undefined` になります）。
- `.eq("id", id)` で「IDが一致する行だけ」に絞り、`.single()` で「配列ではなく1件のオブジェクト」として受け取ります（0件や2件以上だとエラーになります）。

用語: Server Component はサーバー側だけで動くコンポーネントで、DBに直接アクセスできます。**Promise（プロミス）** は「あとで値が決まる箱」で、`await` を付けて中身が出るのを待ちます。

解説2: `null` を空文字に変換して初期値を作る

```
const initialData = {
  // null なら空文字に変換
  title: book.title ?? "",
  author: book.author ?? "",
  publisher: book.publisher ?? "",
  published_date: book.published_date ?? "",
  // null はそのまま
  rating: book.rating ?? null,
  // status は null だと困るのでデフォルトを設定
  status: book.status ?? "unread",
  memo: book.memo ?? "",
};
```

- データベースから取った `book` を、フォームに渡す形（`initialData`）に整え直しています。
- ポイントは `?? ""` で `null` を空文字に変換している点です。React の入力欄に `null` を渡すと「非制御コンポーネント」になってしまい、入力が React 側で管理できなくなるため、文字列の欄はすべて空文字にそろえます。
- `rating` だけは数値か `null` のままで構わない（評価なしを表せる）ので `?? null`、`status` は空だと困るので `?? "unread"` とデフォルトを与えています。

用語: 制御コンポーネント は入力値を React の state で管理する方式で、`value` には必ず文字列（や数値）を渡します。`null` を渡すと**非制御コンポーネント**扱いになり、入力が state に同期されなくなります。

解説3: `BookForm` に props を渡して編集モードで使う

```
<BookForm initialData={initialData} isEdit={true} bookId={id} />
```

- ここがこのページの結論部分です。整えた `initialData`、編集モードを表す `isEdit={true}`、対象のIDである `bookId={id}` の3つを `BookForm` に渡しています。
- これによって `BookForm` は「最初から既存データが入った状態」で表示され、送信時には（`isEdit` が `true` なので）UPDATE を実行します。
- 重いデータ取得はこのサーバー側のページが担当し、対話的な入力処理は受け取った側の `BookForm`（クライアント側）が担当する、という役割分担になっています。

用語: 役割分担（Server/Client の分離） は、データ取得をサーバーコンポーネントで、ユーザー操作をクライアントコンポーネントで行う Next.js App Router の定番パターンです。サーバーからクライアントへは props で値を渡せます。

3. 削除機能

書籍を削除する機能を実装します。削除は取り消しができない操作なので、**確認ダイアログ**を表示してユーザーの意思を再確認してから実行します。

3-1. 削除処理のフロー



削除ボタンを押すと、まず `window.confirm()` による確認ダイアログが表示されます。「本当に "書籍タイトル" を削除しますか？この操作は取り消せません。」というメッセージが表示され、ユーザーが「OK」を選択した場合のみ削除処理が実行されます。「キャンセル」を選択した場合は何も起こりません。

削除確認モダルの選択肢: `window.confirm()` はブラウザ標準のシンプルな確認ダイアログです。お手軽ですが見た目の自由度が低いいため、本格的なプロダクトでは React の状態でモダル（自前の確認画面）を出すパターンも一般的です。今回はシンプルさを優先して `window.confirm` を採用しています。

3-2. DeleteButton コンポーネントの作成

削除ボタンはクライアントサイドのインタラクション（クリックイベント、確認ダイアログ）を必要とするため、`"use client"` ディレクティブを付けたクライアントコンポーネントとして作成します。

ファイル: `components/DeleteButton.tsx`

▼ このコードがやること（先に日本語で）：「削除する」ボタンを作ります。クリックするとまず `window.confirm()` でブラウザ標準の確認ダイアログを出し、ユーザーが「OK」を押したときだけ Supabase の DELETE（削除）を実行します。確認・クリック処理はブラウザ側でしか動かないため、先頭に `"use client"` を付けたクライアントコンポーネントにしています。初心者は「①確認 → ②OKなら削除 → ③一覧へ戻る」という流れと、処理中はボタンを無効化して二重削除を防いでいる点を押さえてください。詳細はコード内のコメントにあります。

```

// クライアントコンポーネント宣言:useState や onClick を使うために必須
"use client";

// 削除中フラグ管理用
import { useState } from "react";
// 削除後の遷移用
import { useRouter } from "next/navigation";
// ブラウザ向け Supabase クライアント
import { createClient } from "@lib/supabase/client";

// -----
// Props の型定義
// -----
// bookId: 削除対象の書籍ID。
// Supabase の DELETE クエリの WHERE 句で
// 使用します。
// bookTitle: 確認ダイアログに表示する書籍タイトル。
// ユーザーが削除対象を確認できるように
// するために使用します。
// -----
type Props = {
  bookId: string;
  bookTitle: string;
};

export default function DeleteButton({ bookId, bookTitle }: Props) {
  // 削除後に一覧へ遷移するために使用
  const router = useRouter();

  // -----
  // 状態管理
  // -----
  // isDeleting: 削除処理中かどうかを管理します。
  // true の間はボタンを無効化し、
  // テキストを「削除中...」に変更して、
  // 二重送信を防ぎます。
  // -----
  const [isDeleting, setIsDeleting] = useState(false);

  // -----
  // 削除処理ハンドラ
  // -----
  // async 関数なので、内部で await を使えます。
  // -----
  const handleDelete = async () => {
    // -----
    // 1. 確認ダイアログの表示
    // -----
    // window.confirm() はブラウザ標準の確認ダイアログ

```

```

// を表示します。ユーザーが「OK」をクリックすると
// true、「キャンセル」をクリックすると false を
// 返します。
//
// 確認ダイアログのメッセージには書籍タイトルを
// 含め、削除対象が明確になるようにします。
// \n で改行を入れています。
// -----
const confirmed = window.confirm(
  `本当に「${bookTitle}」を削除しますか？\nこの操作は取り消せません。`
);

// -----
// キャンセルされた場合は何もしない
// -----
// 早期 return パターン。深いネストを避けて
// 読みやすくする定番の書き方です。
// -----
if (!confirmed) {
  return;
}

// -----
// 2. 削除処理の実行
// -----
// ローディング状態に切り替え
setIsDeleting(true);

try {
  // Supabase クライアントを準備
  const supabase = createClient();

  // -----
  // Supabase DELETE クエリ
  // -----
  // .from("books") で books テーブルを指定し、
  // .delete() で削除操作を行います。
  // .eq("id", bookId) で削除対象を書籍IDで
  // 絞り込みます。
  //
  // ※ .eq() を付けずに .delete() だけを実行すると
  // テーブルの全データが削除されるので、
  // 必ず WHERE 条件を指定してください。
  // (※実際には RLS や Supabase の安全装置で
  // そう簡単には全削除されませんが、習慣として
  // 必ず付けるクセを付けましょう)
  // -----
  const { error } = await supabase
    .from("books")
    // DELETE 文に相当

```

```

.delete()
// WHERE id = bookId
.eq("id", bookId);

if (error) {
  // catch にエラーを渡す
  throw error;
}

// -----
// 3. 削除成功: 一覧ページにリダイレクト
// -----
// 削除した書籍の詳細ページはもう存在しないため、
// 一覧ページに遷移します。
// router.refresh() でサーバーコンポーネントの
// データを再取得し、削除された書籍が一覧から
// 消えていることを確認できるようにします。
// -----
// 一覧ページへ
router.push("/books");
// サーバー側データを再取得
router.refresh();
} catch (err) {
  console.error("削除エラー:", err);
  // -----
  // エラーが発生した場合はアラートで通知
  // -----
  // alert はブラウザ標準のメッセージダイアログ。
  // 簡易的なエラー表示として使っています。
  // -----
  alert("書籍の削除に失敗しました。もう一度お試しください。");
  // 状態を戻してリトライ可能に
  setIsDeleting(false);
}
};

// -----
// ボタンのレンダリング
// -----
// 削除ボタンは赤色で表示し、危険な操作であることを
// 視覚的に示します。
// 削除処理中はボタンを無効化し、テキストを
// 「削除中…」に変更して、処理中であることを
// ユーザーに伝えます。
// -----
return (
  <button
    // クリックで削除ハンドラを呼ぶ
    onClick={handleDelete}
    // 削除中はクリック不可

```

```

disabled={isDeleting}
className="inline-flex items-center px-4 py-2 border border-transparent text-sm
font-medium rounded-md shadow-sm text-white bg-red-600 hover:bg-red-700 focus:outline-
none focus:ring-2 focus:ring-offset-2 focus:ring-red-500 disabled:opacity-50
disabled:cursor-not-allowed"
>
/* 削除中はスピナー+「削除中…」を表示 */
{isDeleting ? (
  <>
    {/* -----
ローディングスピナー
削除処理中に表示される回転アニメーション
です。ユーザーに処理中であることを
視覚的にフィードバックします。
----- */}

<svg
  // animate-spin で回転アニメ
  className="animate-spin -ml-1 mr-2 h-4 w-4 text-white"
  xmlns="http://www.w3.org/2000/svg"
  fill="none"
  viewBox="0 0 24 24"
>
  <circle
    // 25% の透明度で薄く描画
    className="opacity-25"
    cx="12"
    cy="12"
    r="10"
    stroke="currentColor"
    strokeWidth="4"
  />
  <path
    // 75% の透明度でやや濃く描画
    className="opacity-75"
    fill="currentColor"
    // スピナーの一部円弧
    d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 014
12H0c0 3.042 1.135 5.824 3 7.938l3-2.647z"
  />
</svg>
削除中...
</>
) : (
  <>
    {/* ゴミ箱アイコン */}
    <svg
      className="w-4 h-4 mr-2"
      fill="none"
      stroke="currentColor"
      viewBox="0 0 24 24"

```

```

    >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      strokeWidth={2}
      // ゴミ箱の絵
      d="M19 7L-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5
4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16"
    />
  </svg>
  削除する
</>
})
</button>
);
}

```

このコードのポイント:

- `"use client"` ディレクティブが必須です。 `window.confirm()` や `onClick` イベントハンドラはブラウザ側でしか動作しないためです。サーバー側には `window` オブジェクト自体が存在しません。
- `isDeleting` 状態によるローディング管理で、二重クリックによる二重削除を防いでいます。ボタンの `disabled` 属性と組み合わせることで、処理中は追加のクリックを受け付けません。なお DELETE は幂等（何度実行しても結果が同じ）なので、仮に 2 回送られてもデータが壊れることはありませんが、不要なネットワーク負荷を避ける意味でガードします。
- 削除後は `router.push("/books")` で一覧ページに遷移し、 `router.refresh()` でサーバーコンポーネントのキャッシュを更新します。これにより、一覧ページで最新のデータが表示されます。
- エラー時は `alert()` でユーザーに通知し、 `isDeleting` を `false` に戻してリトライ可能にしています。

このボタンは独立した `<button>` であり、フォームの `type="submit"` ではありません。もしフォーム内に置く場合は `type="button"` を明示しないと、親フォームの送信が発火してしまうので注意してください。今回は親が `<form>` ではなくただの `<div>` なのでこの問題は起きません。

▼ コードを1つずつ分解して解説

`DeleteButton` の中で、初心者が押さえておきたい塊を順に見ていきます。

解説1: クリックでまず確認ダイアログを出す

```
const confirmed = window.confirm(
  `本当に「${bookTitle}」を削除しますか？\nこの操作は取り消せません。`
);

if (!confirmed) {
  return;
}
```

- 削除は取り消せない操作なので、いきなり消さずに `window.confirm()`（ブラウザ標準の確認ダイアログ）でユーザーに最終確認します。
- `window.confirm()` は「OK」が押されると `true`、「キャンセル」が押されると `false` を返します。その結果を `confirmed` に入れています。
- `if (!confirmed) { return; }` は「OKでなければ（＝キャンセルなら）ここで処理を打ち切る」という早期 return です。これにより、以降の削除処理はOKのときだけ進みます。
- メッセージに `${bookTitle}` を埋め込み、`\n`（改行）を入れることで「どの本を消すのか」が一目で分かるようにしています。

用語: `window.confirm()` はブラウザに用意された確認ダイアログを出す関数。早期 `return` は「条件を満たさなければ関数を途中で抜ける」書き方で、深い入れ子を避けて読みやすくします。

解説2: 削除中フラグで二重削除を防ぐ

```
// ローディング状態に切り替え
setIsDeleting(true);

try {
  // Supabase クライアントを準備
  const supabase = createClient();

  const { error } = await supabase
    .from("books")
    // DELETE 文に相当
    .delete()
    // WHERE id = bookId
    .eq("id", bookId);

  if (error) {
    // catch にエラーを渡す
    throw error;
  }
}
```

- 削除を始める前に `setIsDeleting(true)` で「削除中」の旗を立てます。この旗が立っている間はボタンを `disabled`（クリック不可）にするので、連打による二重削除を防げます。
- `.delete()` で削除操作を、`.eq("id", bookId)` で「IDが一致する1件だけ」を対象に絞ります。`.eq()` を付けずに `.delete()` だけだと全件削除になりかねないため、必ず条件を付ける習慣をつけます。
- `if (error) { throw error; }` は「Supabase がエラーを返したら、それを `catch` ブロックに投げる」処理です。

用語: `disabled` はボタンを押せなくするHTML属性。`throw` はエラーを発生させて `try/catch` の `catch` 側へ処理を移す命令です。

解説3: 成功時は一覧へ戻し、失敗時はリトライ可能にする

```
// 一覧ページへ
router.push("/books");
// サーバー側データを再取得
router.refresh();
} catch (err) {
  console.error("削除エラー:", err);
  alert("書籍の削除に失敗しました。もう一度お試しください。");
  // 状態を戻してリトライ可能に
  setIsDeleting(false);
}
```

- 削除に成功したら、消した本の詳細ページはもう存在しないので `router.push("/books")` で一覧へ戻します。
- 続く `router.refresh()` は「サーバーコンポーネントのデータを取り直す」命令です。これと呼ばないとキャッシュが残り、一覧に消したはずの本が表示されたままになることがあります。
- 失敗したときは `catch` に入り、`alert` でユーザーに知らせ、`setIsDeleting(false)` で旗を下ろしてもう一度試せる状態に戻します（成功時は遷移するので旗を戻す必要はありません）。

用語: `router.push()` はプログラムからの画面遷移、`router.refresh()` はサーバー側データの再取得を行うメソッド。キャッシュは「一度取得した結果を覚えておく仕組み」で、更新後は明示的に取り直す必要があります。

4. 検索・フィルタ機能（発展）

一覧ページに検索バーとステータスフィルタを追加し、大量の書籍データの中から目的の本を素早く見つけられるようにします。

4-1. 検索バーコンポーネント

タイトルや著者名で書籍を検索できるコンポーネントを作成します。URL の検索パラメータ（ `searchParams` ）を活用して、検索状態をURLに反映させます。これにより、検索結果のURLを共有したり、ブラウザの戻るボタンで検索前の状態に戻ったりできます。

ファイル: `components/SearchBar.tsx`

▼ このコードがやること（先に日本語で）：タイトル・著者名で書籍を絞り込む検索バーと、ステータスで絞り込むフィルタを作ります。検索条件は state ではなく URL の `?q=...&status=...`（検索パラメータ）に反映させるのがポイントで、こうすると検索結果のURLを共有したり、戻るボタンで前の状態に戻したりできます。さらに「キーを打った後に検索せず、入力が止まって300ミリ秒後にまとめて検索する」デバウンスという工夫も入れています。初心者は「入力 → URL を書き換える → サーバーがその条件で再取得する」という流れだけ追えば十分で、`useEffect` やデバウンスの細部はコメントで補足しています。

```

// 入力イベントや setState を扱うのでクライアントコンポーネント
"use client";

// ルーター、URL クエリ、パス取得用フック
import { useRouter, useSearchParams, usePathname } from "next/navigation";
// 状態管理・副作用・関数メモ化
import { useState, useEffect, useCallback } from "react";

// -----
// 検索バーコンポーネント
// -----
// このコンポーネントは以下の機能を提供します:
// 1. テキスト入力による検索
// 2. ステータスによるフィルタリング
// 3. URLの検索パラメータとの同期
// 4. デバウンス処理（入力のたびにクエリを
//    発行しないように、一定時間待ってから検索）
// -----
export default function SearchBar() {
  // URL を変更するため
  const router = useRouter();
  // 現在の ?q=... を読むため
  const searchParams = useSearchParams();
  // 現在のパス（例: /books）を取得
  const pathname = usePathname();

  // -----
  // URL の検索パラメータから初期値を取得
  // -----
  // ページをリロードしたり、検索結果のURLに直接
  // アクセスしたりした場合でも、検索状態が保持
  // されるようにします。
  // -----
  const [searchQuery, setSearchQuery] = useState(
    // ?q= の値、無ければ空文字
    searchParams.get("q") ?? ""
  );
  const [statusFilter, setStatusFilter] = useState(
    // ?status= の値、無ければ空文字
    searchParams.get("status") ?? ""
  );

  // -----
  // URL の検索パラメータを更新する関数
  // -----
  // 検索クエリやフィルタが変更されたときに、
  // URL の検索パラメータを更新してページを
  // 再レンダリングします。
  // useCallback で関数をメモ化することで、

```

```

// 依存配列が変わらない限り同じ関数インスタンスを
// 使い回します。
// -----
const updateSearchParams = useCallback(
  (query: string, status: string) => {
    // 現在のクエリを元にコピーを作成
    const params = new URLSearchParams(searchParams.toString());

    // -----
    // 検索クエリの設定
    // -----
    // 値が空の場合はパラメータを削除し、
    // URL をクリーンに保ちます。
    // 例: /books?q=React&status=reading
    //      検索クエリが空なら /books?status=reading
    // -----
    // 空白以外の文字が含まれていれば
    if (query.trim()) {
      // ?q= を設定
      params.set("q", query.trim());
    } else {
      // 空なら ?q= を削除
      params.delete("q");
    }

    // ステータスフィルタの設定
    if (status) {
      params.set("status", status);
    } else {
      params.delete("status");
    }

    // -----
    // URL を更新
    // -----
    // router.push() を使って URL を更新すると、
    // Next.js が自動的にサーバーコンポーネントを
    // 再レンダリングし、新しい searchParams で
    // データを再取得します。
    // -----
    // "q=React&status=reading" など
    const queryString = params.toString();
    // クエリがあれば付加、なければパスだけ
    const newUrl = queryString ? `${pathname}?${queryString}` : pathname;
    // URL を更新
    router.push(newUrl);
  },
  // これらが変わったら関数を作り直す
  [router, pathname, searchParams]
);

```

```

// -----
// デバウンス処理
// -----
// ユーザーがキーボードを打つたびに検索クエリを
// 発行すると、不必要なリクエストが大量に発生
// します。デバウンスを使って、ユーザーが入力を
// 止めてから 300ms 後に検索を実行します。
// useEffect は「描画後に副作用を実行する」フック。
// -----
useEffect(() => {
  // 300ms 後に実行する予約
  const timer = setTimeout(() => {
    updateSearchParams(searchQuery, statusFilter);
  }, 300);

  // -----
  // クリーンアップ関数
  // -----
  // 次の入力が行われた場合、前のタイマーを
  // キャンセルします。これにより、最後の入力
  // から 300ms 後にのみ検索が実行されます。
  // -----
  // 副作用の片付け
  return () => clearTimeout(timer);
}, [searchQuery, statusFilter, updateSearchParams]);

// -----
// 検索条件クリア
// -----
// 入力欄を空にして、URL も検索条件無しに戻します。
// -----
const handleClear = () => {
  // 検索クエリをクリア
  setSearchQuery("");
  // フィルタをクリア
  setStatusFilter("");
  // ?xxx を全部外す
  router.push(pathname);
};

// -----
// レンダリング
// -----
return (
  <div className="bg-white shadow rounded-lg p-4 mb-6">
    {/* モバイルは縦、PC は横並び */}
    <div className="flex flex-col sm:flex-row gap-4">
      {/* 検索入力フィールド */}

```

```

<div className="flex-1">
  {/* sr-only は視覚的には非表示、スクリーンリーダー用 */}
  <label htmlFor="search" className="sr-only">
    検索
  </label>
  <div className="relative">
    {/* 虫眼鏡アイコン */}
    {/* クリックを通過させる */}
    <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-
events-none">
      <svg
        className="h-5 w-5 text-gray-400"
        fill="none"
        stroke="currentColor"
        viewBox="0 0 24 24"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          strokeWidth={2}
          // 虫眼鏡のパス
          d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
        />
      </svg>
    </div>
    <input
      type="text"
      id="search"
      // state を反映（制御コンポーネント）
      value={searchQuery}
      // 入力のたびに state を更新
      onChange={(e) => setSearchQuery(e.target.value)}
      placeholder="タイトルまたは著者名で検索..."
      className="block w-full pl-10 pr-3 py-2 border border-gray-300 rounded-md
leading-5 bg-white placeholder-gray-500 focus:outline-none focus:placeholder-gray-400
focus:ring-1 focus:ring-blue-500 focus:border-blue-500 sm:text-sm"
    />
  </div>
</div>

{/* ステータスフィルタ */}
<div className="sm:w-48">
  <label htmlFor="status-filter" className="sr-only">
    ステータスフィルタ
  </label>
  <select
    id="status-filter"
    value={statusFilter}
    onChange={(e) => setStatusFilter(e.target.value)}
    className="block w-full py-2 px-3 border border-gray-300 bg-white rounded-

```

```

md shadow-sm focus:outline-none focus:ring-blue-500 focus:border-blue-500 sm:text-sm"
  >
    <option value="">すべてのステータス</option>
    <option value="unread">未読</option>
    <option value="reading">読書中</option>
    <option value="finished">読了</option>
  </select>
</div>

{/* クリアボタン */}
/* どちらか入っているときだけ表示 */
{(searchQuery || statusFilter) && (
  <button
    onClick={handleClear}
    className="inline-flex items-center px-3 py-2 border border-gray-300 text-
sm font-medium rounded-md text-gray-700 bg-white hover:bg-gray-50 focus:outline-none
focus:ring-2 focus:ring-offset-2 focus:ring-blue-500"
  >
    <svg
      className="w-4 h-4 mr-1"
      fill="none"
      stroke="currentColor"
      viewBox="0 0 24 24"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        strokeWidth={2}
        // × アイコン
        d="M6 18L18 6M6 6l12 12"
      />
    </svg>
    クリア
  </button>
)}
</div>
</div>
);
}

```

▼ コードを1つずつ分解して解説

`SearchBar` には、初心者がつまづきやすい「URL同期」と「デバウンス」という工夫が入っています。塊ごとに見ていきましょう。

解説1: 初期値を URL の検索パラメータから取る

```
const [searchQuery, setSearchQuery] = useState(  
  // ?q= の値、無ければ空文字  
  searchParams.get("q") ?? ""  
);  
const [statusFilter, setStatusFilter] = useState(  
  // ?status= の値、無ければ空文字  
  searchParams.get("status") ?? ""  
);
```

- 検索条件を state（変化する値）として持ちますが、その初期値を URL から読み込んでいるのがポイントです。
- `searchParams.get("q")` は URL の `?q=...` の値を取り出します。値が無いときは `null` が返るので、`?? ""`（左が `null/undefined` なら右）で空文字にそろえています。
- こうしておく、検索結果のURLを直接開いたりページをリロードしたりしても、入力欄に前回の検索条件が復元されます。

用語: `searchParams` は URL の `?` 以降（クエリ文字列）を読むためのオブジェクト。`.get("q")` で `?q=` の値を取り出せます。

解説2: 入力をデバウンスして「打ち終わってから」検索する

```
useEffect(() => {  
  // 300ms 後に実行する予約  
  const timer = setTimeout(() => {  
    updateSearchParams(searchQuery, statusFilter);  
  }, 300);  
  
  // 副作用の片付け  
  return () => clearTimeout(timer);  
  // これらが変わるたびに発火  
}, [searchQuery, statusFilter, updateSearchParams]);
```

- もしキーを打つたびに検索すると、不要なリクエストが大量に飛んでしまいます。それを防ぐのがデバウンスという工夫です。
- `useEffect`（描画後に副作用を行うフック）の中で `setTimeout` を使い、「300ミリ秒後に検索する」という予約をします。
- 戻り値の `return () => clearTimeout(timer)` が肝です。次のキー入力があると、まずこの後片付けで前の予約をキャンセルしてから新しい予約を入れ直します。結果として「入力が止まって300ミリ秒経ったとき」だけ検索が走ります。

用語: デバウンス (debounce) は「連続するイベントを間引いて、最後の1回だけ処理する」テクニック。**クリーンアップ関数** (`useEffect` の `return`) は、次の実行前や後片付けのときに呼ばれる掃除用の関数です。

解説3: 空の条件は URL から削除してきれいに保つ

```
// 空白以外の文字が含まれていれば
if (query.trim()) {
  // ?q= を設定
  params.set("q", query.trim());
} else {
  // 空なら ?q= を削除
  params.delete("q");
}
```

- 検索条件を URL に書き込むとき、**値があるときだけ付け、空なら消す**ことで URL を読みやすく保っています。
- `query.trim()` で前後の空白を除き、中身があれば `params.set("q", ...)` で `?q=...` を設定します。
- 空のときは `params.delete("q")` で `?q=` を消します。これにより `/books?status=reading` のように「使っている条件だけ」が URL に残ります。

用語: URLSearchParams はクエリ文字列を組み立て・編集するための道具で、`.set()` で項目を追加/上書き、`.delete()` で項目を削除できます。

4-2. 一覧ページの更新 (検索・フィルタ対応)

検索バーコンポーネントを一覧ページに組み込み、Supabase のクエリを検索条件に対応させます。

ファイル: `app/books/page.tsx` (更新版)

▼ このコードがやること (先に日本語で) : 前章で作った一覧ページを、検索・フィルタ・ソートに対応させた更新版です。URL の検索パラメータ (`q` / `status` / `sort` / `order`) を読み取り、その条件に合わせて Supabase へのクエリを少しずつ組み立てていきます。`ilike` は大文字小文字を区別しない部分一致検索、`.or()` は「タイトル または 著者名」のどちらかにマッチさせる指定です。初心者は「URL の条件を見て → クエリに条件を足していく → 最後にまとめて実行する」という組み立て方に注目してください。各条件の詳細はコード内のコメントで説明しています。


```

// サーバー用 Supabase クライアント
import { createClient } from "@lib/supabase/server";
// 詳細ページ等への遷移
import Link from "next/link";
// 検索バー (クライアントコンポーネント)
import SearchBar from "@components/SearchBar";
// ソート用セレクト (クライアントコンポーネント)
import SortSelect from "@components/SortSelect";

// -----
// 型定義
// -----
// Next.js App Router のページコンポーネントは、
// searchParams プロパティでURLの検索パラメータを
// 受け取ることができます。
// Next.js 15 以降は Promise でラップされる点に注意。
// -----
type Props = {
  // URL の ? 以降のパラメータ
  searchParams: Promise<{
    // 検索クエリ (省略可)
    q?: string;
    // ステータスフィルタ
    status?: string;
    // ソート対象カラム
    sort?: string;
    // 昇順/降順
    order?: string;
  }>;
};

// -----
// ステータス表示用のヘルパー関数
// -----
// 詳細ページで使ったものと同じ。
// 共通化したい場合は別ファイルに抽出するとよい。
// -----
function getStatusLabel(status: string): string {
  const statusMap: Record<string, string> = {
    unread: "未読",
    reading: "読書中",
    finished: "読了",
  };
  return statusMap[status] || status;
}

function getStatusColor(status: string): string {
  switch (status) {
    case "unread":

```

```

    return "bg-gray-100 text-gray-800";
  case "reading":
    return "bg-blue-100 text-blue-800";
  case "finished":
    return "bg-green-100 text-green-800";
  default:
    return "bg-gray-100 text-gray-800";
}
}

// -----
// 評価を星マークで表示する関数
// -----
function renderStars(rating: number | null): string {
  // 未評価のときはハイフン
  if (rating === null || rating === undefined) return "-";
  return "★".repeat(rating) + "☆".repeat(5 - rating);
}

// -----
// 書籍一覧ページ (Server Component)
// -----
export default async function BooksPage({ searchParams }: Props) {
  // Promise を解決して各パラメータを取り出し
  const { q, status, sort, order } = await searchParams;
  // サーバー用クライアント
  const supabase = await createClient();

  // -----
  // Supabase クエリの構築
  // -----
  // 基本のクエリを作成し、検索条件やフィルタ条件が
  // ある場合は動的にクエリを組み立てます。
  // let で宣言して、後から条件を追加していく形にします。
  // -----
  // ベースのクエリ (全件取得)
  let query = supabase.from("books").select("*");

  // -----
  // テキスト検索
  // -----
  // q パラメータが指定されている場合、タイトルまたは
  // 著者名で部分一致検索を行います。
  //
  // Supabase の ilike は大文字小文字を区別しない
  // LIKE 検索です。% はワイルドカードで、
  // %検索語% は「検索語を含む」という意味になります。
  //
  // .or() を使って、タイトルまたは著者名のいずれかに
  // マッチする条件を設定します。

```

```

// -----
if (q) {
  // title OR author で部分一致
  query = query.or(`title.ilike.%%${q}%,author.ilike.%%${q}%`);
}

// -----
// ステータスフィルタ
// -----
// status パラメータが指定されている場合、
// そのステータスの書籍のみを取得します。
// -----
if (status) {
  // WHERE status = :status
  query = query.eq("status", status);
}

// -----
// ソート
// -----
// sort パラメータでソート対象のカラムを、
// order パラメータでソート順（昇順/降順）を
// 指定します。デフォルトは作成日の降順です。
// -----
// 指定が無ければ created_at
const sortColumn = sort || "created_at";
// "asc" のときだけ昇順
const ascending = order === "asc";
// ORDER BY :sortColumn :order
query = query.order(sortColumn, { ascending });

// -----
// クエリ実行
// -----
// ここで初めてサーバーへリクエストが飛ぶ。
// -----
const { data: books, error } = await query;

if (error) {
  console.error("書籍の取得に失敗:", error);
}

// -----
// レンダリング
// -----
return (
  <div className="max-w-4xl mx-auto py-8 px-4">
    {/* ページヘッダー */}
    <div className="flex items-center justify-between mb-6">
      <h1 className="text-2xl font-bold text-gray-900">書籍一覧</h1>

```

```

<Link
  href="/books/new"
  className="inline-flex items-center px-4 py-2 border border-transparent text-
sm font-medium rounded-md shadow-sm text-white bg-blue-600 hover:bg-blue-700"
>
  <svg
    className="w-4 h-4 mr-2"
    fill="none"
    stroke="currentColor"
    viewBox="0 0 24 24"
  >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      strokeWidth={2}
      // + アイコン
      d="M12 4v16m8-8H4"
    />
  </svg>
  新規登録
</Link>
</div>

{/* 検索バーとソート */}
<SearchBar />
<SortSelect />

{/* 検索結果の件数表示 */}
<p className="text-sm text-gray-500 mb-4">
  {books ? `${books.length}件の書籍が見つかりました` : "読み込み中..."}
  {q && (
    <span className="ml-2">
      (検索: 「{q}」)
    </span>
  )}
  {status && (
    <span className="ml-2">
      (フィルタ: {getStatusLabel(status)})
    </span>
  )}
</p>

{/* 書籍一覧 */}
/* データが無いとき */
{!books || books.length === 0 ? (
  <div className="text-center py-12 bg-white rounded-lg shadow">
    <svg
      className="mx-auto h-12 w-12 text-gray-400"
      fill="none"
      stroke="currentColor"

```

```

    viewBox="0 0 24 24"
  >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      strokeWidth={2}
      // 開いた本のアイコン
      d="M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5 5S4.168 5.477 3
6.253v13C4.168 18.477 5.754 18 7.5 18s3.332.477 4.5 1.253m0-13C13.168 5.477 14.754 5
16.5 5c1.747 0 3.332.477 4.5 1.253v13C19.832 18.477 18.247 18 16.5 18c-1.746 0-
3.332.477-4.5 1.253"
    />
  </svg>
  <h3 className="mt-2 text-sm font-medium text-gray-900">
    書籍が見つかりません
  </h3>
  <p className="mt-1 text-sm text-gray-500">
    {q || status
      ? "検索条件を変更してお試しください。"
      : "新しい書籍を登録してみましょう。"}
  </p>
  /* 検索条件無しで 0 件のときだけ「最初の書籍を登録」を表示 */
  {!q && !status && (
    <div className="mt-6">
      <Link
        href="/books/new"
        className="inline-flex items-center px-4 py-2 border border-transparent
shadow-sm text-sm font-medium rounded-md text-white bg-blue-600 hover:bg-blue-700"
      >
        最初の書籍を登録する
      </Link>
    </div>
  )}
</div>
) : (
  <div className="bg-white shadow overflow-hidden sm:rounded-lg">
    <ul className="divide-y divide-gray-200">
      /* 配列を map で展開してリスト要素を生成 */
      {books.map((book) => (
        /* key は React がリスト要素を識別するために必要 */
        <li key={book.id}>
          <Link
            // クリックで詳細ページへ
            href={`\books/${book.id}`}
            className="block hover:bg-gray-50 transition-colors duration-150"
          >
            <div className="px-4 py-4 sm:px-6">
              <div className="flex items-center justify-between">
                <div className="flex-1 min-w-0">
                  <p className="text-sm font-medium text-blue-600 truncate">

```

```

        {book.title}
      </p>
      <p className="mt-1 text-sm text-gray-500">
        {book.author || "著者不明"}
        {/* 出版社があれば併記 */}
        {book.publisher && ` / ${book.publisher}`}
      </p>
    </div>
    <div className="ml-4 flex flex-col items-end space-y-1">
      <span
        className={`inline-flex items-center px-2.5 py-0.5 rounded-
full text-xs font-medium ${getStatusColor(book.status)}`}
      >
        {getStatusLabel(book.status)}
      </span>
      {book.rating && (
        <span className="text-yellow-500 text-xs">
          {renderStars(book.rating)}
        </span>
      )}
    </div>
  </div>
</div>
</Link>
</li>
)}}
</ul>
</div>
)}
</div>
);
}

```

検索・フィルタ機能のポイント:

- Supabase の `ilike` メソッドは、PostgreSQL の `ILIKE` 演算子に対応しており、大文字小文字を区別しない部分一致検索を行います。`%` はワイルドカードで、`%React%` は「React」を含むすべての文字列にマッチします。
- `.or()` メソッドを使うことで、「タイトルに含まれる または 著者名に含まれる」という OR 条件を設定できます。
- URLの検索パラメータ（`searchParams`）を使うことで、検索状態がURLに反映されます。ブラウザの戻る/進むボタンで検索状態を行き来したり、検索結果のURLを他の人と共有したりできます。

▼ コードを1つずつ分解して解説

更新版の一覧ページでは、URL の条件を見ながらクエリを少しずつ組み立てていきます。塊ごとに見ていきましょう。

解説1: クエリを `let` で宣言して後から条件を足す

```
// ベースのクエリ (全件取得)
let query = supabase.from("books").select("*");
```

- まず「全件取得」という土台のクエリを作り、`let` (あとで再代入できる変数) に入れておきます。
- このあと検索やフィルタの条件があるときだけ、この `query` に条件を継ぎ足していきます。
- ポイントは、この時点ではまだサーバーに問い合わせしていないことです。Supabase のクエリは「組み立てている途中」で、最後に `await` したときに初めて実行されます。

用語: `let` は再代入できる変数の宣言 (`const` は再代入不可)。クエリビルダーは「条件をメソッドでつないで組み立て、最後に実行する」仕組みのことです。

解説2: 検索とフィルタの条件を継ぎ足す

```
if (q) {
  // title OR author で部分一致
  query = query.or(`title.ilike.%${q}%,author.ilike.%${q}%`);
}

if (status) {
  // WHERE status = :status
  query = query.eq("status", status);
}
```

- `if (q)` で「検索語があるときだけ」テキスト検索の条件を足します。`.or(...)` は「タイトル または 著者名」のどちらかにマッチさせる指定です。
- `ilike` は大文字小文字を区別しない部分一致検索で、`%${q}%` の `%` (ワイルドカード) が「前後に何があってもよい=含む」を表します。
- `if (status)` も同様に、ステータスが選ばれているときだけ `.eq("status", status)` (status が一致する行だけ) を足します。条件が無ければ何も足さないなので、全件のままになります。

用語: `ilike` は大文字小文字を無視する部分一致検索 (PostgreSQL の `ILIKE`)。 `%` (ワイルドカード) は「任意の文字列」を表す記号で、`%語%` は「その語を含む」という意味になります。

```
// 指定が無ければ created_at
const sortColumn = sort || "created_at";
// "asc" のときだけ昇順
const ascending = order === "asc";
// ORDER BY :sortColumn :order
query = query.order(sortColumn, { ascending });

const { data: books, error } = await query;
```

- 並び順を決めます。`sort || "created_at"` は「URL に並び替え対象が無ければ登録日（`created_at`）を使う」という意味です（`||` は左が空なら右）。
- `order === "asc"` で「URL の order が "asc" のときだけ昇順（小さい順）」と判定し、それ以外は降順になります。
- `.order(sortColumn, { ascending })` で並び順を足し、最後の `await query` でここで初めてサーバーへ問い合わせます。組み立て終えたクエリをまとめて実行する流れです。

用語: `.order()` は並び替えの指定（SQL の `ORDER BY`）。昇順（`ascending`）は小さい順・古い順、降順（`descending`）は大きい順・新しい順を指します。

5. ソート機能

書籍一覧をさまざまな基準で並べ替えられるソートコンポーネントを作成します。

ファイル: `components/SortSelect.tsx`

▼ このコードがやること（先に日本語で）：一覧の並び順を切り替えるセレクトボックスを作ります。選択肢の値は `"title-asc"` のように「カラム名-昇順/降順」をハイフンでつないだ形式にしておき、選ばれたら2つに分解して URL の `?sort=...&order=...` に書き込みます。検索バーと同じく、並び順も URL に持たせることで共有や戻る操作に対応できます。初心者は「選択 → 値を分解 → URL を更新 → サーバーがその順番で取り直す」という流れを押さえてください。詳細はコード内のコメントにあります。


```

// セレクト変更時に router.push を呼ぶのでクライアントコンポーネント
"use client";

// ルーティング関連フック
import { useRouter, useSearchParams, usePathname } from "next/navigation";

// -----
// ソート選択コンポーネント
// -----
// 書籍一覧の並び順を変更するためのセレクトボックスです。
// 以下のソート基準に対応しています:
// - 登録日（新しい順/古い順）
// - タイトル（昇順/降順）
// - 評価（高い順/低い順）
// -----

export default function SortSelect() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const pathname = usePathname();

  // -----
  // 現在のソート状態をURLから取得
  // -----
  // URL に sort/order が無ければデフォルト値を使う。
  // -----
  // デフォルトは登録日
  const currentSort = searchParams.get("sort") || "created_at";
  // デフォルトは降順
  const currentOrder = searchParams.get("order") || "desc";
  // select の value 形式に変換
  const currentValue = `${currentSort}-${currentOrder}`;

  // -----
  // ソート変更ハンドラ
  // -----
  // セレクトボックスの値が変更されたときに呼ばれます。
  // 値は "カラム名-昇順/降順" の形式（例: "title-asc"）
  // になっており、これを分解して URL パラメータに
  // 設定します。
  // -----

  const handleSortChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
    // 例: "rating-desc"
    const value = e.target.value;
    // ["rating", "desc"] に分解
    const [sort, order] = value.split("-");

    // 現在のクエリをコピー
    const params = new URLSearchParams(searchParams.toString());
    // sort を上書き

```

```

params.set("sort", sort);
// order を上書き
params.set("order", order);

// 新しい URL に遷移
router.push(`${pathname}?${params.toString()}`);
};

// -----
// レンダリング
// -----
return (
  <div className="flex items-center justify-end mb-4">
    <label
      htmlFor="sort"
      className="text-sm text-gray-500 mr-2"
    >
      並び順:
    </label>
    <select
      id="sort"
      // URL の値を反映
      value={currentValue}
      // 変更時に URL を更新
      onChange={handleSortChange}
      className="block w-48 py-1.5 px-3 border border-gray-300 bg-white rounded-md shadow-sm focus:outline-none focus:ring-blue-500 focus:border-blue-500 text-sm"
    >
      <option value="created_at-desc">登録日（新しい順） </option>
      <option value="created_at-asc">登録日（古い順） </option>
      <option value="title-asc">タイトル（A→Z） </option>
      <option value="title-desc">タイトル（Z→A） </option>
      <option value="rating-desc">評価（高い順） </option>
      <option value="rating-asc">評価（低い順） </option>
    </select>
  </div>
);
}

```

ソート機能のポイント:

- Supabase の `.order()` メソッドは、PostgreSQL の `ORDER BY` 句に対応しています。第1引数にカラム名、第2引数のオブジェクトで `ascending: true` を指定すると昇順、`ascending: false`（または省略）で降順になります。
- ソート条件もURLの検索パラメータに反映されるため、検索・フィルタと同様にブラウザの履歴管理やURL共有が可能です。
- 検索・フィルタとソートは独立して動作します。例えば「未読」フィルタをかけた状態で「評価（高い順）」にソートすると、「未読かつ評価の高い順」で表示されます。

▼ コードを1つずつ分解して解説

`SortSelect` の中で、初心者が押さえておきたい塊を順に見ていきます。

解説1: URL から現在の並び順を組み立てて select に反映する

```
// デフォルトは登録日
const currentSort = searchParams.get("sort") || "created_at";
// デフォルトは降順
const currentOrder = searchParams.get("order") || "desc";
// select の value 形式に変換
const currentValue = `${currentSort}-${currentOrder}`;
```

- セレクトボックスに「今どの並び順が選ばれているか」を表示するため、現在の状態を URL から読み取ります。
- `searchParams.get("sort") || "created_at"` は「URL に `?sort=` が無ければ登録日（`created_at`）を初期値にする」という意味です。`order` も同様にデフォルトは降順（`desc`）です。
- 取り出した2つを ``${currentSort}-${currentOrder}`` でハイフン区切りにつなぎ、`"created_at-desc"` のような形にします。これは下の `<option value="...">` の値と同じ形式で、`select` の `value` に渡して選択状態を一致させるためです。

用語: テンプレートリテラル（``...``）はバッククォートで囲み、`${変数}` で値を文字列に埋め込める書き方。
`select` の `value` に現在値を渡すと、その値に一致する `option` が選択された状態で表示されます。

解説2: 選ばれた値を分解して URL に書き込む

```
const handleSortChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  // 例: "rating-desc"
  const value = e.target.value;
  // ["rating", "desc"] に分解
  const [sort, order] = value.split("-");

  // 現在のクエリをコピー
  const params = new URLSearchParams(searchParams.toString());
  // sort を上書き
  params.set("sort", sort);
  // order を上書き
  params.set("order", order);

  // 新しい URL に遷移
  router.push(`${pathname}?${params.toString()}`);
};
```

- セレクトの値が変わったときに呼ばれる関数です。選ばれた値（例: `"rating-desc"`）を `.split("-")` でハイフンの位置で2つに割り、`[sort, order]` の分割代入で `sort="rating"` , `order="desc"` のように取り出します。
- `new URLSearchParams(searchParams.toString())` で今の URL の条件をコピーしてから `.set()` で `sort` と `order` だけを上書きします。こうすることで、検索やフィルタの条件を消さずに並び順だけを差し替えられます。
- 最後に `router.push(...)` で新しい URL に遷移すると、サーバーがその並び順でデータを取り直します。

用語: `.split("-")` は文字列を区切り文字で分割して配列にするメソッド。分割代入（`[sort, order] = ...`）は配列の要素を一度に別々の変数へ取り出す書き方です。

6. 全機能の動作確認

ここまでで実装した CRUD 全操作と検索・フィルタ・ソート機能の動作を確認します。以下の手順に沿って、各機能を順番にテストしてください。

6-1. 新規登録のテスト

1. ブラウザで `http://localhost:3000/books` にアクセスします
2. 右上の「新規登録」ボタンをクリックします
3. `http://localhost:3000/books/new` に遷移したことを確認します
4. 以下の情報を入力します: - タイトル: `React実践ガイド` - 著者: `山田太郎` - 出版社: `技術評論社` - 出版日: `2024-01-15` - 評価: `★★★★☆ (4)` - ステータス: `読書中` - メモ: `コンポーネント設計の章が特に参考になった`
5. 「登録する」ボタンをクリックします
6. 期待される結果: 一覧ページにリダイレクトされ、登録した書籍が一覧に表示されていること
7. 同様に、もう2冊登録します: - タイトル: `TypeScript入門`, 著者: `鈴木花子`, ステータス: `未読`, 評価: `3` - タイトル: `Next.js実践`, 著者: `田中一郎`, ステータス: `読了`, 評価: `5`

6-2. 詳細表示のテスト

1. 一覧ページで「React実践ガイド」をクリックします
2. 詳細ページに遷移したことを確認します

3. 期待される結果: - URL が `/books/{書籍ID}` の形式であること - タイトル「React実践ガイド」がヘッダーに表示されていること - 著者「山田太郎」がヘッダーの下に表示されていること - ステータス「読書中」のバッジが青色で表示されていること - 出版社「技術評論社」が表示されていること - 出版日「2024年1月15日」が日本語形式で表示されていること - 評価「★★★★☆」が黄色い星で表示されていること - メモの内容が灰色の背景内に表示されていること - 「編集する」ボタン（青色）と「削除する」ボタン（赤色）が表示されていること
4. 「一覧に戻る」リンクをクリックして、一覧ページに戻れることを確認します

6-3. 編集のテスト

1. 詳細ページに戻り、「編集する」ボタンをクリックします
2. 編集ページに遷移したことを確認します
3. 期待される結果: - URL が `/books/{書籍ID}/edit` の形式であること - フォームの各フィールドに既存のデータが入った状態で表示されていること
 - タイトル欄に「React実践ガイド」
 - 著者欄に「山田太郎」
 - 出版社欄に「技術評論社」
 - 出版日が「2024-01-15」
 - 評価が「★★★★☆（4）」
 - ステータスが「読書中」
 - メモ欄に「コンポーネント設計の章が特に参考になった」
 - ボタンのテキストが「更新する」であること
4. 以下の変更を行います: - 評価を「★★★★★（5）」に変更 - ステータスを「読了」に変更 - メモに「最後まで読了。全体的に素晴らしい内容だった」を追加
5. 「更新する」ボタンをクリックします
6. 期待される結果: 詳細ページにリダイレクトされ、更新された情報が反映されていること - 評価が「★★★★★」になっていること - ステータスが「読了」（緑色のバッジ）に変わっていること - メモが更新されていること

6-4. 検索・フィルタのテスト

1. 一覧ページに戻ります
2. 検索バーに「React」と入力します
3. 期待される結果: 「React実践ガイド」のみが表示されること - URL に `?q=React` が含まれていること - 「1件の書籍が見つかりました（検索:「React」）」と表示されること
4. 検索バーをクリアし、ステータスフィルタで「未読」を選択します
5. 期待される結果: ステータスが「未読」の書籍のみが表示されること - URL に `?status=unread` が含まれていること
6. 「クリア」ボタンをクリックして検索条件をリセットします
7. 期待される結果: 全書籍が表示されること

6-5. ソートのテスト

1. ソートのセレクトボックスで「評価（高い順）」を選択します
2. **期待される結果:** 評価の高い書籍から順に表示されること
3. 「タイトル（A→Z）」を選択します
4. **期待される結果:** タイトルのアルファベット/五十音順に表示されること

6-6. 削除のテスト

1. 一覧ページから「TypeScript入門」をクリックして詳細ページに移動します
2. 「削除する」ボタンをクリックします
3. **期待される結果:** 確認ダイアログが表示され、「本当に「TypeScript入門」を削除しますか？この操作は取り消せません。」というメッセージが表示されること
4. まず「キャンセル」をクリックします
5. **期待される結果:** 何も変化せず、詳細ページが表示されたままであること
6. 再度「削除する」ボタンをクリックし、今度は「OK」をクリックします
7. **期待される結果:** - 一覧ページにリダイレクトされること - 「TypeScript入門」が一覧から消えていること - 残りの書籍が正しく表示されていること

7. トラブルシューティング

開発中によく遭遇するエラーとその対処法をまとめます。

7-1. params の取得でエラーが発生する

エラーメッセージ:

```
Error: Cannot read properties of undefined (reading 'id')
```

または

```
TypeError: params.then is not a function
```

原因と対処法:

Next.js のバージョンによって `params` の扱いが異なります。

- Next.js 15 以降: `params` は `Promise` として渡されるため、`await params` で解決する必要があります。
- Next.js 14 以前: `params` は通常のオブジェクトとして渡されるため、直接 `params.id` でアクセスします。

```
// Next.js 15 以降
type Props = {
  // Promise でラップされている
  params: Promise<{ id: string }>;
};

export default async function Page({ params }: Props) {
  // await が必要
  const { id } = await params;
  // ...
}

// Next.js 14 以前
type Props = {
  // ただのオブジェクト
  params: { id: string };
};

export default async function Page({ params }: Props) {
  // await 不要
  const { id } = params;
  // ...
}
```

自分のプロジェクトの Next.js バージョンは `package.json` で確認できます。

```
# package.json から "next" を含む行を抜き出して表示
cat package.json | grep next
```

7-2. UPDATE / DELETE が反映されない

症状: UPDATE や DELETE を実行してもエラーは出ないが、データが変更されていない。

原因1: RLS (Row Level Security) ポリシーが設定されていない、または不適切

Supabase では RLS がデフォルトで有効になっています。テーブルに適切なポリシーが設定されていない場合、操作が静かに無視されることがあります。

対処法: Supabase のダッシュボードで RLS ポリシーを確認します。

```
-- Supabase SQL Editor で実行
-- 現在のポリシーを確認
-- システムビューから books テーブル用のポリシーを一覧表示
SELECT * FROM pg_policies WHERE tablename = 'books';
```

開発中は、以下のような全許可ポリシーを設定できます（本番環境では適切な認証ベースのポリシーに変更してください）。

▼ このコードがやること（先に日本語で）：Supabase の SQL Editor で実行する、開発用の「全部許可」ルール（ポリシー）を作る SQL です。Supabase はセキュリティ機能（RLS）により、ルールが無いと読み書きが静かに無視されてしまうため、学習中だけ「誰でも全操作OK」のルールを置いて動くようにします。**USING (true)** と **WITH CHECK (true)** の **true** が「常に許可」を意味します。初心者は「これは開発専用で、本番では必ず認証ベースのルールに置き換える」点だけ覚えておいてください。

```
-- すべての操作を許可するポリシー（開発用）
-- ポリシー名を "Allow all operations" として books テーブルに作成
CREATE POLICY "Allow all operations" ON books
-- SELECT/INSERT/UPDATE/DELETE すべてに適用
FOR ALL
-- 既存行へのアクセス条件：常に true（全許可）
USING (true)
-- 新規追加/更新時のチェック条件：常に true（全許可）
WITH CHECK (true);
```

▼ コードを1つずつ分解して解説

この開発用ポリシー（RLS のルール）を、行ごとに分解して見ていきます。

解説1: ポリシーを作る対象と適用範囲

```
-- ポリシー名を "Allow all operations" として books テーブルに作成
CREATE POLICY "Allow all operations" ON books
-- SELECT/INSERT/UPDATE/DELETE すべてに適用
FOR ALL
```

- **CREATE POLICY "名前" ON テーブル名** で、指定したテーブル（ここでは **books**）に対するアクセスルールを1つ作ります。**"Allow all operations"** は後から見分けるためのポリシー名です。
- **FOR ALL** は「このルールを SELECT（読む）/INSERT（追加）/UPDATE（更新）/DELETE（削除）のすべての操作に適用する」という指定です。

用語: RLS（Row Level Security、行レベルセキュリティ） は「テーブルの行ごとにアクセス可否を制御する」Supabase/PostgreSQL の仕組み。**ポリシー** はその許可ルール1件のことです。

解説2: `USING (true)` と `WITH CHECK (true)` の意味

```
-- 既存行へのアクセス条件: 常に true (全許可)
USING (true)
-- 新規追加/更新時のチェック条件: 常に true (全許可)
WITH CHECK (true);
```

- `USING (...)` は「すでにある行を読む/更新/削除してよいか」を判定する条件です。`true` (常に真) なので「どの行でもOK」になります。
- `WITH CHECK (...)` は「これから追加/更新する行を許してよいか」を判定する条件です。こちらも `true` なので「どんな書き込みもOK」になります。
- つまりこのポリシーは「誰でも全操作OK」という最も緩い設定です。学習中に手早く動かすため専用で、**本番では必ず** 認証ベースのルールに置き換えます (次の 7-4 で本番例を扱います)。

用語: `USING` は既存行へのアクセス条件、`WITH CHECK` は書き込み内容の検査条件。`true` は「常に許可」を意味します。

原因2: `.eq()` の条件が合っていない

UPDATE や DELETE で `.eq("id", bookId)` の `bookId` が正しくない可能性があります。

対処法: コンソールログで値を確認します。

```
// 渡ってきた bookId を確認
console.log("Updating book with ID:", bookId);
const { data, error } = await supabase
  .from("books")
  .update({ title: "..." })
  .eq("id", bookId)
  // .select() を付けて更新後のデータを返してもらう
  .select();
// data が [] なら該当 ID 無し、null なら RLS で弾かれている可能性
console.log("Update result:", { data, error });
```

原因3: `router.refresh()` を呼んでいない

Next.js のサーバーコンポーネントはデータをキャッシュするため、データベースを更新しただけでは画面に反映されません。

対処法: `router.push()` の後に `router.refresh()` を呼びます。

```
router.push("/books");
// サーバーコンポーネントのキャッシュを更新（最新データで再取得）
router.refresh();
```

Server Action を使っている場合は **revalidatePath** を使う: **revalidatePath("/books")** を呼ぶと、指定した URL のキャッシュが破棄され、次のアクセスで最新データが取れます。Server Action 内では **router.refresh** が呼べないので、こちらを使います。

7-3. リダイレクトが動作しない

症状: **router.push()** を呼んでもページが遷移しない。

原因1: サーバーコンポーネント内で **useRouter** を使っている

useRouter はクライアントコンポーネント専用のフックです。サーバーコンポーネント内では使用できません。

対処法: サーバーコンポーネントでリダイレクトしたい場合は **redirect()** 関数を使います。

```
// サーバーコンポーネント内でのリダイレクト
// サーバー側遷移用の関数
import { redirect } from "next/navigation";

export default async function Page() {
  // 何らかの条件でリダイレクト
  // この行で関数全体が中断され、ブラウザは /books に飛ぶ
  redirect("/books");
}
```

原因2: **"use client"** ディレクティブの付け忘れ

useRouter や **useState** などの React フックを使用するコンポーネントには **"use client"** ディレクティブが必要です。

```
// ファイルの最初に追加（最上行・1 行目に書く）
"use client";

import { useRouter } from "next/navigation";
// ...
```

原因3: try-catch の catch ブロック内で処理が止まっている

エラーが発生して catch ブロックに入ると、リダイレクト処理が実行されません。コンソールでエラーを確認してください。

```
try {
  // ... Supabase 操作
  // エラーが発生するところまで到達しない
  router.push("/books");
} catch (err) {
  // コンソールでエラーを確認
  console.error("Error:", err);
}
```

7-4. RLS 関連のエラー

エラーメッセージ:

```
{
  code: "42501",
  message: "new row violates row-level security policy for table \"books\""
}
```

原因: RLS ポリシーが書き込み操作を許可していません。

対処法:

1. Supabase ダッシュボードの「Authentication」>「Policies」で、**books** テーブルのポリシーを確認します。
2. 開発段階では、以下の SQL で全操作を許可できます。

▼ このコードがやること（先に日本語で）：先ほどの「全部許可」ポリシーを作り直す SQL です。同じ名前のポリシーが既にあったら **CREATE** でエラーになるため、先に **DROP POLICY IF EXISTS** で「あれば消す」を実行してから作り直しています（**IF EXISTS** のおかげで、無くてもエラーになりません）。初心者は「ポリシーを入れ替えたいときは、いったん消してから作り直す」という2段階のやり方として覚えておけば大丈夫です。

```

-- 既存のポリシーを削除（必要に応じて）
-- 同名ポリシーがあれば削除（IF EXISTS でエラー回避）
DROP POLICY IF EXISTS "Allow all operations" ON books;

-- 全操作許可ポリシーを作成
-- ポリシーを再作成
CREATE POLICY "Allow all operations" ON books
  -- すべての操作対象
  FOR ALL
  -- 全行を許可
  USING (true)
  -- 全書き込みを許可
  WITH CHECK (true);

```

3. 本番環境では、認証されたユーザーのみが操作できるようにポリシーを設定します。

▼このコードがやること（先に日本語で）：本番向けの、安全なルール（ポリシー）の例です。ログイン済みのユーザー（`authenticated`）だけを対象にし、さらに `auth.uid() = user_id` で「自分が登録したデータだけ操作できる」ように制限します（`auth.uid()` は今ログインしている人のIDです）。初心者は「本番では『誰でもOK』ではなく『ログイン済みかつ自分のデータだけ』に絞る」という考え方を押さえてください。なお、この例は `books` テーブルに `user_id` 列がある前提で、本章の現段階では未実装なので、今は開発用ポリシーを使います。

```

-- 認証済みユーザーのみ操作を許可
-- ポリシー名は説明的に
CREATE POLICY "Authenticated users can manage books" ON books
  -- すべての操作
  FOR ALL
  -- 認証済みユーザー（ロール authenticated）のみ
  TO authenticated
  -- 既存行の所有者チェック
  USING (auth.uid() = user_id)
  -- 書き込み時の所有者チェック
  WITH CHECK (auth.uid() = user_id);

```

▼コードを1つずつ分解して解説

開発用の「全部許可」と違い、本番用は「ログイン済みかつ自分のデータだけ」に絞ります。塊ごとに見ていきましょう。

解説1: 適用対象を「認証済みユーザー」に限定する

```
-- ポリシー名は説明的に
CREATE POLICY "Authenticated users can manage books" ON books
-- すべての操作
FOR ALL
-- 認証済みユーザー（ロール authenticated）のみ
TO authenticated
```

- **books** テーブルに対するルールを作る点は開発用と同じですが、名前を「何をするポリシーか」が分かる説明的なものにしています。
- 開発用に無かった **TO authenticated** が肝です。これは「ログイン済みのユーザー（**authenticated** ロール）にだけこのルールを適用する」という指定で、未ログインの訪問者は対象外になります。

用語: **ロール (role)** は Supabase における利用者の種別。**authenticated** は「ログイン済みユーザー」、**anon** は「未ログイン（匿名）ユーザー」を表します。**TO authenticated** で対象をログイン済みに絞れます。

解説2: 「自分のデータだけ」に制限する所有者チェック

```
-- 既存行の所有者チェック
USING (auth.uid() = user_id)
-- 書き込み時の所有者チェック
WITH CHECK (auth.uid() = user_id);
```

- 開発用では **true**（常に許可）だった部分を、**auth.uid() = user_id** という条件に変えています。
- **auth.uid()** は「今ログインしている人のユーザーID」を返す関数です。これが行の **user_id** 列と一致するとき、つまり自分が登録したデータのときだけ許可します。
- **USING** 側で「自分の行だけ読む/更新/削除できる」を、**WITH CHECK** 側で「他人のIDを付けて保存できない」を保証します。これにより、ユーザーごとにデータが安全に分離されます。

用語: **auth.uid()** は現在ログイン中のユーザーIDを返す Supabase の関数。**所有者チェック** は「そのデータの持ち主だけが操作できる」ようにする仕組みです。なお、この例は **books** テーブルに **user_id** 列がある前提で、本章の現段階では未実装です。

注意: 上記の本番用ポリシーは、**books** テーブルに **user_id** カラムがあり、ユーザーごとにデータを分離する場合の例です。このチュートリアル of 現段階では **user_id** カラムは未実装なので、開発用の全許可ポリシーを使用してください。

7-5. フォームの初期値が反映されない

症状: 編集ページでフォームを開いても、フィールドが空になっている。

原因: `useState` の初期値が正しく設定されていない可能性があります。

対処法:

1. 編集ページで `initialData` が正しく渡されているか確認します。

```
// app/books/[id]/edit/page.tsx
// DB から取れた生データ
console.log("Book data from DB:", book);
// フォームに渡す整形後データ
console.log("Initial data for form:", initialData);
```

2. `BookForm` コンポーネントで `initialData` を受け取れているか確認します。

```
// components/BookForm.tsx
// 受け取った props
console.log("Received initialData:", initialData);
// 内部状態
console.log("Form state:", formData);
```

3. `null` 値の扱いに注意します。データベースから取得した値が `null` の場合、フォームの入力フィールドが uncontrolled になる可能性があります。

```
// null を空文字に変換
const initialData = {
  // null の場合は空文字 (?? は null/undefined のときだけ右の値)
  title: book.title ?? "",
  // null の場合は空文字
  author: book.author ?? "",
  // ...
};
```

8. 全体のページ遷移図

最後に、書籍管理アプリ全体のページ遷移を確認します。以下の図は、ユーザーがアプリ内でどのようにページ間を移動するかを示しています。

書籍管理アプリ - ページ遷移図



主要な遷移パス

- 一覧 → 新規登録 → 一覧 (登録完了/キャンセル)
- 一覧 → 詳細 → 編集 → 詳細 (更新完了/キャンセル)
- 詳細 → 一覧 (「一覧に戻る」/ 削除完了)

この遷移図から、アプリの主要な導線を確認できます。

- 一覧 → 新規登録 → 一覧: 新しい書籍を登録する流れ。登録完了後は一覧ページに戻り、登録した書籍がリストに追加されていることを確認できます。
- 一覧 → 詳細 → 編集 → 詳細: 書籍の情報を編集する流れ。一覧から詳細を確認し、「編集する」ボタンで編集ページへ。編集完了後は詳細ページに戻り、更新内容を確認できます。
- 一覧 → 詳細 → 削除 → 一覧: 書籍を削除する流れ。詳細ページで「削除する」ボタンを押すと確認ダイアログが表示され、確認後に削除が実行されて一覧ページに戻ります。

まとめ

この章では、以下の機能を実装しました。

機能	ファイル	説明
詳細表示	<code>app/books/[id]/page.tsx</code>	書籍の全情報を表示するページ
編集	<code>app/books/[id]/edit/page.tsx</code>	既存データをフォームに表示して更新
削除	<code>components/DeleteButton.tsx</code>	確認ダイアログ付きの削除ボタン
フォーム（共通）	<code>components/BookForm.tsx</code>	新規登録と編集の両方に対応
検索・フィルタ	<code>components/SearchBar.tsx</code>	テキスト検索とステータスフィルタ
ソート	<code>components/SortSelect.tsx</code>	複数のソート基準に対応

これで、書籍管理アプリの CRUD 機能がすべて揃いました。ユーザーは書籍の登録、表示、編集、削除を一通り行えるようになり、検索やフィルタ、ソートで効率的にデータを管理できます。

次の章では、認証機能を追加してユーザーごとにデータを分離する方法を学びます。